Rack-Level Micro-Climate & Growth Optimizer

INDIVIDUAL THESIS

H.P.D.M.Chandrasiri

IT22180148

Individual component: Biological Stage Detection & Harvest Prediction

Group ID: R26 – ISE-002

Report submitted in partial fulfilment of the requirements for the

BSC (Hons) Information Technology

Information System Engineering

Department of Computer System Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

2026 June

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: [Full name]

Student ID: [Student ID]

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Narmada Gamage		
Co-supervisor: Rangi Liyanage		

ABSTRACT

Oyster mushroom farmers often depend on manual observation to monitor mushroom growth and identify suitable harvesting times. This approach can require frequent inspections and may make it difficult to monitor multiple cultivation racks consistently. Although computer vision provides opportunities for automated agricultural monitoring, reliable growth-stage detection and practical harvest-time information remain important challenges under varying cultivation conditions. This research aims to develop a Biological Stage Detection and Harvest Prediction component for the Rack-Level Micro-Climate & Growth Optimizer for Oyster Mushroom Farms. The proposed component uses images captured by a fixed camera positioned near cultivation racks and a YOLOv8 object detection model to identify four oyster mushroom growth stages: early growth, pinning, mature, and ready-to-harvest. The model was trained for 50 epochs and achieved a reported mean Average Precision at an Intersection over Union threshold of 0.50 (mAP50) of approximately 92.2% on the evaluation dataset. The software architecture incorporates a FastAPI backend, MongoDB for monitoring records, and a React Native Android application intended to present rack-level information and estimated remaining time until harvest. The component is designed to complement environmental monitoring by providing visual information about mushroom development. The reported detection result indicates the model's potential for image-based growth monitoring; however, further evaluation under practical cultivation conditions is required to establish its generalizability. Harvest-time prediction also requires separate validation against actual harvesting records. The individual contribution focuses on the design and development of the biological growth-stage detection component, YOLOv8 model training and evaluation, and integration planning for rack-level monitoring and farmer-oriented information presentation. The proposed component provides a foundation for reducing reliance on repeated manual observation and supporting more informed oyster mushroom harvesting decisions.

Keywords: Oyster mushroom, YOLOv8, growth-stage detection, harvest prediction, computer vision

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my research supervisor for providing valuable guidance, advice, and continuous support throughout my individual research and development work on Biological Stage Detection and Harvest Prediction for Oyster Mushroom Farms. The feedback and recommendations provided by my supervisor helped me improve my research approach and develop a better understanding of the research area.

I would also like to thank the lecturers and academic staff of SLIIT for the knowledge and support provided throughout my undergraduate studies. Their guidance has contributed to my academic and technical development.

Furthermore, I would like to acknowledge my project team members for their cooperation and support in working towards the overall research project, Rack-Level Micro-Climate & Growth Optimizer for Oyster Mushroom Farms. Their collaboration has supported the integration of my individual component with the other components of the project.

Finally, I am grateful to my family and everyone who encouraged and supported me throughout this research journey. Their patience, understanding, and encouragement motivated me to continue working towards the successful completion of my research.

This research was completed using the resources available to me. Any additional technical assistance or funding, where applicable, will be acknowledged accordingly.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 1

1.3 Problem statement 2

1.4 Research gap 3

1.5 Aim and objectives 3

1.6 Research questions or hypotheses 4

1.7 Scope limitations and assumptions 4

1.8 Individual contribution 5

1.9 Thesis structure 6

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review method and scope 7

2.2 Domain and theoretical foundations 8

2.3 Related research 10

2.4 Existing systems products and techniques 13

2.5 Gap synthesis and conceptual framework 18

3 RESEARCH METHODOLOGY 22

3.1 Research design ..... **Error! Bookmark not defined.**

3.2 Data participants or study materials ..... **Error! Bookmark not defined.**

3.3 Experimental or study procedure ..... **Error! Bookmark not defined.**

3.4 Measures and evaluation metrics ..... **Error! Bookmark not defined.**

3.5 Data analysis ..... **Error! Bookmark not defined.**

3.6 Ethics privacy security and responsible AI ..... **Error! Bookmark not defined.**

3.7 Reproducibility and research artefacts ..... **Error! Bookmark not defined.**

4 PRODUCT AND SOLUTION DESIGN 35
4.1 Requirements 35
4.2 Component architecture within the complete solution 37
4.3 Design alternatives and decisions 40
4.4 Detailed design 43
4.5 Standards and professional practice 49
5 IMPLEMENTATION INTEGRATION AND TESTING 53
5.1 Development environment and technology stack 53
5.2 Component implementation 55
5.3 Integration with the other components 59
5.4 Verification and software testing 59
5.5 Deployment and operation 64
6 RESULTS AND VALIDATION 68
6.1 Research results 68
6.2 Product and technical results 68
6.3 User or stakeholder validation 69
6.4 Integration results 70
6.5 Summary of findings 71
7 DISCUSSION 72
7.1 Interpretation of findings 72
7.2 Comparison with literature baselines and existing solutions 72
7.3 Research contribution 73
7.4 Product contribution 74
7.5 Limitations and threats to validity 75
7.6 Implications 76

8 COMMERCIALISATION AND SUSTAINABILITY **Error! Bookmark not defined.**

8.1 Value proposition and target users ..... **Error! Bookmark not defined.**

8.2 Market and competitive position..... **Error! Bookmark not defined.**

8.3 Business or sustainability model..... **Error! Bookmark not defined.**

8.4 Scalability intellectual property and compliance **Error! Bookmark not defined.**

8.5 Contribution to the Sustainable Development Goals **Error! Bookmark not defined.**

9 CONCLUSION AND FUTURE DIRECTIONS 78

9.1 Conclusion 78

9.2 Achievement of objectives 78

9.3 Individual research and product contributions 80

9.4 Future directions..... 81

REFERENCES..... 83

GLOSSARY..... 84

APPENDICES 85

Appendix A Research instruments or extended results..... 85

Appendix B Academic integrity and AI use disclosure 85

TEMPLATE CONVENTIONS ..... **Error! Bookmark not defined.**

LIST OF FIGURES

[Guidance - delete before submission: Use Word's Insert Caption command with the label Figure. Number figures by chapter and cite every figure in the text.]

No table of figures entries found.

LIST OF TABLES

[Guidance - delete before submission: Use Word's Insert Caption command with the label Table. Number tables by chapter and cite every table in the text.]

No table of figures entries found.

LIST OF ABBREVIATIONS

AI	*Artificial Intelligence*
API	*Application Programming Interface*
CNN	*Convolutional Neural Network*
CPU	*Central Processing Unit*
CV	*Computer Vision*
DL	*Deep Learning*
FPS	*Frames Per Second*
HTTP	*Hypertext Transfer Protocol*
IoT	*Internet of Things*
JSON	*JavaScript Object Notation*
mAP	*Mean Average Precision*
ML	*Machine Learning*
MQTT	*Message Queuing Telemetry Transport*
REST	*Representational State Transfer*
RGB	*Red, Green, Blue*
SSD	*Solid-State Drive*
UI	*User Interface*
URL	*Uniform Resource Locator*
YOLO	*You Only Look Once*

1 INTRODUCTION

1.1 Overall project context

Oyster mushroom farming is becoming an important agricultural activity because oyster mushrooms can be cultivated using relatively low-cost materials and limited space. However, successful mushroom cultivation requires proper monitoring of environmental conditions and mushroom growth. Small-scale and beginner farmers may face difficulties when monitoring mushroom racks manually, especially when many cultivation bags are present at different growth stages.

The overall research project, Rack-Level Micro-Climate & Growth Optimizer for Oyster Mushroom Farms aims to develop a smart agricultural solution that supports oyster mushroom farmers in monitoring their cultivation environment and mushroom growth. The system combines sensing, data processing, image analysis, and intelligent prediction to provide useful information for better farm management.

The project focuses particularly on rack-level monitoring because conditions and mushroom growth can differ between racks and even between cultivation bags. By collecting and analyzing information at rack level, the system can provide more specific information to farmers instead of giving only general information about the entire farm.

Within the overall project, several functions are considered, including environmental monitoring and mushroom growth analysis. This research focuses specifically on the Biological Stage Detection & Harvest Prediction component.

1.2 Individual component context

The individual component of this research is Biological Stage Detection & Harvest Prediction The purpose of this component is to automatically analyze oyster mushroom images and identify their biological growth stage. The system uses images captured through a fixed camera positioned at the mushroom cultivation rack.

The component uses computer vision and deep learning techniques to analyze visible mushroom characteristics such as size, shape, colour, and cap development. A

YOLOv8 based object detection model is used to recognize the different growth stages of oyster mushrooms.

The biological stages considered in this component are early growth, pinning, mature, and ready-to-harvest However, the biological stage names are mainly used internally by the prediction model. The farmer facing system is designed to provide more practical information, particularly the estimated time remaining until harvestor an indication that the mushrooms are ready for harvesting.

The component therefore aims to reduce the need for continuous manual observation and provide farmers with timely information about mushroom development and harvesting readiness.

1.3 Problem statement

Oyster mushroom farmers need to regularly observe mushroom growth to determine when the mushrooms are developing properly and when they are ready to harvest. In traditional farming practices, this monitoring is mainly performed through manual visual observation. Farmers need to inspect cultivation bags and estimate the growth stage based on their experience.

Manual monitoring can become difficult when the number of cultivation bags increases. Different mushrooms may also develop at different rates, making it difficult to provide an accurate harvest estimate through occasional observation. In addition, beginner farmers may not have enough experience to distinguish between different stages of mushroom development.

Another problem is that manual observations may not be recorded consistently. As a result, farmers may have difficulty tracking the development of mushrooms over time and identifying the appropriate harvesting period.

Therefore, there is a need for an automated image-based system that can analyze mushroom growth and provide useful harvest-related information to farmers. This research addresses this problem by developing a computer vision-based biological stage detection and harvest prediction component using YOLOv8.

1.4 Research gap

Existing mushroom cultivation practices commonly depend on manual observation and the experience of farmers to identify mushroom growth and harvesting readiness. Although image processing and deep learning techniques have been increasingly used in agricultural applications, there is still a need for practical solutions that can support rack-level monitoring of oyster mushrooms, particularly for beginner and small-to-medium-scale farmers.

A further gap is the connection between biological stage detection and practical harvesting information. Simply identifying a biological stage may not be sufficiently useful for a farmer. Farmers are more interested in knowing when the mushrooms are likely to be ready for harvesting.

Therefore, this research focuses not only on detecting mushroom growth stages but also on using the detected growth information to provide a practical estimated time until harvest. The component also considers a fixed camera approach rather than depending on farmers manually taking photographs using their mobile phones. This can support more consistent monitoring of mushroom growth.

1.5 Aim and objectives

Aim

The main aim of this research component is to develop an automated computer vision-based system that detects the biological growth stages of oyster mushrooms and provides an estimated harvest readiness prediction using images captured from cultivation racks.

Objectives

The objectives of this component are:

1. To collect and prepare suitable oyster mushroom images representing different biological growth stages.
2. To develop and train a YOLOv8 based deep learning model for detecting oyster mushroom growth stages.
3. To evaluate the performance of the trained model using suitable evaluation measures such as accuracy and mAP.

4. To integrate the trained model with a backend system for processing mushroom images.
5. To develop a system that can provide rack-level mushroom growth information based on captured images.
6. To estimate the remaining time until the mushrooms are ready for harvesting based on the detected growth condition.
7. To provide simple and understandable harvest-related information to farmers rather than displaying complex biological stage information.
8. To evaluate the usability and effectiveness of the component for beginner and small-to-medium-scale oyster mushroom farmers.

1.6 Research questions or hypotheses

The research is guided by the following research questions:

1. How accurately can a YOLOv8 based computer vision model detect the biological growth stages of oyster mushrooms?
2. Can image-based biological stage detection be used to provide a useful estimation of oyster mushroom harvest readiness?
3. Can a fixed camera system support consistent rack-level monitoring of oyster mushroom growth?
4. How effectively can the proposed system reduce the difficulty of manually monitoring mushroom growth?

5. Is the proposed harvest prediction information understandable and useful for beginner and small-to-medium-scale oyster mushroom farmers?

1.7 Scope limitations and assumptions

- Scope

This research focuses on the automated detection of oyster mushroom biological growth stages using images captured from cultivation racks. The system considers four main growth stages as early growth, pinning, mature, and ready to harvest.

The component includes image acquisition, image preprocessing, YOLOv8 model training, biological stage detection, model evaluation, backend integration, and harvest

readiness prediction. The system is designed to operate at rack level and provide information related to the estimated time until harvest.

The target users are mainly beginner and small to-medium scale oyster mushroom farmers who may require additional support in monitoring mushroom growth.

- Limitations

The accuracy of the system can be affected by image quality, lighting conditions, camera position, mushroom orientation, overlapping mushrooms, and differences between mushroom varieties or cultivation conditions.

The model performance also depends on the quality and diversity of the training dataset. If the training dataset does not contain sufficient examples of certain growth conditions, the model may have difficulty recognizing those conditions accurately.

The harvest prediction is an estimation and cannot guarantee the exact harvesting date because mushroom development can be affected by environmental and biological factors.

This component focuses on oyster mushrooms and does not aim to detect every possible mushroom species or agricultural crop.

- Assumptions

The system assumes that the fixed camera can capture sufficiently clear images of the mushrooms. It also assumes that the camera is positioned appropriately to provide useful views of the cultivation area.

It is assumed that the images used for model training are correctly labelled according to the relevant biological growth stages. The research also assumes that the mushroom cultivation environment follows reasonably consistent cultivation practices during system evaluation.

1.8 Individual contribution

The individual contribution of this research is the design and development of the Biological Stage Detection & Harvest Prediction component of the overall oyster mushroom smart farming system.

The main contribution is the development of a computer vision-based approach for identifying oyster mushroom growth stages using a YOLOv8 deep learning model. The component also connects biological stage detection with practical harvest-related information by providing an estimated time until the mushrooms are ready to harvest.

The contribution includes the preparation and organization of image data, model training and evaluation, development of the image analysis process, integration of the trained model with the backend, and development of the harvest prediction logic.

The component also contributes a rack-level monitoring approach using a fixed camera. This reduces the dependence on farmers manually capturing and uploading individual mushroom images and supports more continuous monitoring.

The trained model achieved approximately 92.2% mAP50 during evaluation, demonstrating the potential of deep learning-based image analysis for oyster mushroom growth monitoring.

1.9 Thesis structure

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The literature review for this research component was conducted to identify existing methods, technologies, and research findings related to oyster mushroom growth-stage detection and harvest-time prediction. The review focused on computer vision, deep learning, object detection models, and artificial intelligence applications in mushroom cultivation. Particular attention was given to the YOLO family of object detection models because these models can identify and locate objects in images and support automated visual monitoring.

The review also considered studies related to mushroom growth-stage classification, image-based agricultural monitoring, and AI-based harvesting predictions. Different approaches were examined to understand their methods, advantages, limitations, and suitability for practical mushroom farming. The findings from the reviewed literature were used to guide the selection of YOLOv8 for detecting oyster mushroom growth stages and developing a system that estimates the time remaining until harvest.

Review Scope

The scope of the literature review was limited to image-based oyster mushroom growth monitoring and harvest prediction. It covered the identification of four growth stages: early growth, pinning, mature, and ready to harvest. The review examined how deep learning models can analyze mushroom images and support automated monitoring in cultivation environments.

The review also considered the practical requirements of small and medium-sized oyster mushroom farms in Sri Lanka. These requirements include reducing manual observation, supporting rack-wise monitoring, providing timely harvest estimates, and making the system easy for farmers to use. The proposed solution uses a fixed camera to capture mushroom images rather than depending on farmers to upload photographs manually. YOLOv8 is used for growth-stage detection, while the wider system uses FastAPI, MongoDB, and a React Native Android application to support processing, record management, and user interaction.

However, this review does not focus on temperature and humidity prediction, mushroom disease detection, or general microclimate optimization, as these are outside the primary scope of this component. The main objective is to evaluate existing image-based detection approaches and identify opportunities to improve oyster mushroom growth monitoring and estimated harvest-time prediction.

2.2 Domain and theoretical foundations

Domain Foundations

This research component belongs to the fields of smart agriculture, artificial intelligence, computer vision, and deep learning. Oyster mushroom cultivation requires regular monitoring to identify mushroom growth and determine the appropriate harvesting time. Traditionally, farmers observe mushroom bags manually to check the size, shape, color, and development of mushroom caps. However, manual monitoring can be time-consuming and may lead to differences in judgment between farmers. Therefore, an automated image-based monitoring system can help improve the consistency and efficiency of mushroom cultivation.

Computer vision is an important technology in this research because it enables computers to analyze and understand images. By using images captured through a fixed camera installed near the mushroom racks, the proposed system can automatically analyze the visible characteristics of oyster mushrooms. This approach supports continuous rack-wise monitoring without requiring farmers to capture and upload images manually.

Theoretical Foundations

1. Artificial Intelligence (AI)

Artificial intelligence refers to the ability of computer systems to perform tasks that normally require human intelligence. In this research, AI is used to analyze oyster mushroom images and identify their growth stages. The system aims to reduce the need for continuous manual observation and provide useful information for harvesting decisions.

2. Computer Vision

Computer vision enables computers to extract meaningful information from digital images. In oyster mushroom cultivation, visible characteristics such as mushroom size, cap shape, color, and cluster development can provide information about growth. The proposed system uses computer vision to analyze images captured at mushroom racks and identify the relevant growth stages.

3. Deep Learning and Convolutional Neural Networks

Deep learning is a branch of machine learning that uses multiple layers of artificial neural networks to learn patterns from data. Convolutional Neural Networks (CNNs) are commonly used in image-processing tasks because they can learn visual features such as edges, shapes, textures, and patterns. In this research, deep learning supports the identification of oyster mushroom growth stages from captured images.

4. YOLOv8 Object Detection Model

You Only Look Once version 8 (YOLOv8) is a deep-learning model used for object detection and related computer vision tasks. Object detection can identify objects in an image and locate them using bounding boxes. In this research, a YOLOv8 model is trained to recognize four oyster mushroom growth-stage classes: early growth, pinning, mature, and ready to harvest. The trained model provides the visual detection results used by the wider system to support growth monitoring.

5. Growth-Stage Classification and Harvest-Time Prediction

Growth-stage classification involves assigning observed mushrooms to predefined growth categories based on their visual characteristics. Identifying the growth stage provides useful information about mushroom development. Harvest-time prediction goes a step further by estimating how much time remains before the mushrooms are ready to harvest. However, detecting a growth stage does not automatically provide an accurate harvest-time estimate. A suitable prediction method must be developed and validated using relevant observations, image records, and harvesting data.

6. Smart Agriculture and Decision Support

Smart agriculture uses digital technologies, sensors, automation, and data analysis to improve agricultural activities. The proposed system supports smart agriculture by

providing automated visual monitoring and estimated harvest-time information. The system is intended to help small and medium-sized oyster mushroom farmers in Sri Lanka monitor their cultivation racks and plan harvesting activities more effectively.

2.3 Related research

Recent developments in artificial intelligence, computer vision, and deep learning have created new opportunities for automated mushroom growth monitoring. Researchers have investigated different approaches to identify mushroom growth stages, detect mushroom fruiting bodies, and support harvesting decisions. These studies provide a foundation for developing automated oyster mushroom monitoring systems. However, differences in mushroom varieties, cultivation environments, detection methods, and prediction requirements create opportunities for further research.

2.3.1 Oyster Mushroom Growth-Stage Identification

Guo et al. (2023) investigated oyster mushroom growth-stage identification using computer vision technologies. Their study introduced a dataset containing images of three oyster mushroom growth stages and developed a monitoring system incorporating image acquisition, cloud storage, label mapping, and application-based monitoring. The researchers evaluated different computer vision approaches and reported that the VGG-16 model achieved an accuracy of 82.22%. Their findings demonstrated that computer vision could be used to identify oyster mushroom growth stages in images captured in actual cultivation environments. However, the study focused primarily on growth-stage identification and did not establish a complete system for predicting the remaining time until harvest. This creates an opportunity to investigate the integration of growth-stage detection with harvest-time estimation.

2.3.2 Deep Learning for Mushroom Detection

Chen et al. (2023) proposed an improved YOLOv5s-based algorithm for detecting white button mushrooms (Agaricus bisporus) in mushroom-growing environments. The study focused on improving mushroom detection to support harvesting robots. The researchers demonstrated the potential of deep learning-based object detection for

recognizing mushrooms in cultivation environments. This research is relevant because it illustrates how YOLO-based models can support automated mushroom detection. However, white button mushroom detection and oyster mushroom growth-stage identification are different tasks. A model designed to detect mushroom fruiting bodies does not automatically identify their growth stages or predict their harvesting time. Therefore, a model must be trained and evaluated for the specific requirements of oyster mushroom cultivation.

2.3.3 Mushroom Growth-Stage Detection and Harvest-Time Prediction

Fan et al. (2026) proposed a framework called YOLO-Pulmonarius for mushroom growth-stage detection and maturity prediction. Their research integrated an improved YOLOv12-based detection model with logistic growth modelling and recursive Gaussian fusion to estimate the time required for mushrooms to reach full maturity. The model was trained using 3,867 annotated images covering three growth stages of grey oyster mushrooms. The researchers reported a mean average precision at an intersection-over-union threshold of 0.50 (mAP@50) of 97.2% and a mean absolute error of 0.42 days for maturity-time prediction. This study is particularly relevant because it combines visual growth-stage detection with temporal maturity prediction. However, its approach uses a different model architecture and growth-stage configuration from the YOLOv8-based approach investigated in the present research. Its reported performance also needs to be interpreted within the context of its own dataset and experimental conditions.

2.3.4 Computer Vision for Mushroom Growth Monitoring

Wang et al. (2026) investigated a multitask deep learning approach for oyster mushroom growth management through segmentation and depth prediction. Their research introduced MushroomSegDepth, a model that combines mushroom segmentation with depth estimation from RGB images. The approach aims to obtain information about mushroom structure and growth, providing additional measurements that may help researchers understand mushroom development over time. This research highlights the value of extracting measurable visual information

rather than relying only on discrete growth-stage categories. However, segmentation and depth estimation do not directly provide an estimate of the remaining time until harvest. Additional temporal analysis and validation are needed to translate visual measurements into reliable harvesting predictions.

2.3.5 Research Gap and Relevance to the Proposed System

The reviewed studies demonstrate that computer vision and deep learning can support automated mushroom detection and growth monitoring. Guo et al. (2023) investigated oyster mushroom growth-stage identification, Chen et al. (2023) explored YOLO-based mushroom detection, Fan et al. (2026) combined growth-stage detection with maturity-time prediction, and Wang et al. (2026) investigated segmentation and depth estimation for mushroom growth management.

Although these studies provide valuable foundations, the approaches differ in their objectives, model architectures, datasets, and prediction methods. Therefore, their reported results cannot be compared directly without considering these differences. In particular, a detection model's performance does not necessarily demonstrate that its harvest-time predictions are accurate.

The present research focuses on developing a YOLOv8-based system for identifying four oyster mushroom growth stages: early growth, pinning, mature, and ready to harvest. The proposed system is intended to use images captured by a fixed camera installed near the mushroom racks, supporting rack-wise monitoring without requiring farmers to upload photographs manually. The wider system integrates a FastAPI backend, MongoDB database, and React Native Android application.

The intended contribution is to connect automated growth-stage detection with practical harvest-time estimation in a system designed for small and medium-sized oyster mushroom farms in Sri Lanka. The usefulness of this approach will depend not only on the detection model's performance but also on the accuracy of its harvest estimates, the reliability of image capture, and the system's performance under actual cultivation conditions.

2.4 Existing systems products and techniques

Several systems and techniques have been developed to improve mushroom cultivation through automation, computer vision, and artificial intelligence. Existing approaches include manual growth monitoring, image-based mushroom detection, deep learning models, and automated agricultural monitoring systems. These approaches provide useful foundations for developing an automated oyster mushroom growth-monitoring system. However, their suitability depends on their ability to identify growth stages, estimate harvesting time, and operate under practical mushroom cultivation conditions.

2.4.1 Manual Mushroom Growth Monitoring

Manual observation is a traditional method used by mushroom farmers to monitor mushroom development and determine the appropriate harvesting time. Farmers visually inspect mushroom bags and observe characteristics such as cap size, shape, color, and overall development. This method does not require expensive equipment or advanced technical knowledge.

However, manual monitoring requires regular attention and can become time-consuming when farms contain many cultivation bags arranged across multiple racks. Harvesting decisions may also vary according to the experience and judgment of individual farmers. In addition, farmers may find it difficult to monitor every rack consistently. These limitations create an opportunity for an automated system that analyzes mushroom images and provides consistent monitoring information.

2.4.2 Image-Based Mushroom Detection Systems

Image-based mushroom detection systems use cameras and computer vision techniques to identify mushrooms in captured images. These systems can help locate individual mushrooms, detect mushroom clusters, and estimate visible characteristics. Some approaches are designed to support automated harvesting equipment, while others focus on monitoring mushroom growth.

Deep learning models can recognize mushrooms under different lighting conditions and in complex cultivation environments, depending on their training data and model design. However, detecting the presence of a mushroom is different from identifying its growth stage. A system that detects mushrooms may not determine whether they

are in early growth, pinning, maturity, or ready for harvesting. Therefore, models must be trained and evaluated according to the specific requirements of the intended application.

2.4.3 YOLO-Based Object Detection Techniques

You Only Look Once (YOLO) is a family of deep learning models widely used for real-time object detection. YOLO models can identify objects and locate them within images using bounding boxes. Their relatively efficient processing makes them suitable for applications that require rapid image analysis.

Different YOLO versions have been investigated for agricultural applications, including fruit detection, crop monitoring, and mushroom detection. For example, researchers have explored YOLO-based methods to detect white button mushrooms and support automated harvesting. These studies demonstrate the potential of YOLO models for recognizing agricultural products in cultivation environments.

The proposed research uses YOLOv8 to identify four oyster mushroom growth-stage classes: early growth, pinning, mature, and ready to harvest. Unlike a system designed only to detect mushroom locations, this component focuses on identifying growth-stage categories that can support harvest planning. However, the model's effectiveness depends on the quality and diversity of the training dataset, image conditions, and its performance during testing.

2.4.4 Deep Learning-Based Growth-Stage Classification

Deep learning techniques, including Convolutional Neural Networks (CNNs), can learn visual patterns from labeled images. These patterns may include mushroom size, shape, color, texture, and cap development. A trained model can use the learned patterns to classify images or detect objects belonging to predefined categories.

Traditional image-processing techniques may rely on manually defined features and thresholds. Deep learning methods can learn more complex visual representations directly from training images. Nevertheless, deep learning systems require suitable datasets, sufficient computational resources, and careful evaluation to reduce incorrect predictions.

For oyster mushroom cultivation, growth-stage classification can provide useful information about mushroom development. However, the model must distinguish between visually similar stages and account for differences in mushroom size, cluster density, lighting, and camera position. These factors are important when designing a reliable monitoring system for practical farm environments.

2.4.5 Harvest-Time Prediction Techniques

Harvest-time prediction techniques aim to estimate when an agricultural product will reach the appropriate harvesting condition. These approaches may use historical growth observations, environmental measurements, image features, mathematical growth models, or machine learning algorithms.

In mushroom cultivation, detecting the current growth stage can provide information about the development of the mushrooms. However, growth-stage detection alone does not determine the exact number of days remaining until harvest. An additional prediction method is required to estimate the remaining time. Such a method should be evaluated using actual growth observations and recorded harvesting times.

The proposed system aims to use image-based growth-stage detection to support harvest-time estimation. The intended output is an estimated time remaining until harvest, presented in a farmer-friendly format. The reliability of this functionality depends on the prediction method implemented and the availability of suitable validation data.

2.4.6 Fixed-Camera and Rack-Level Monitoring Systems

Fixed-camera monitoring systems capture images from a specific location without requiring a person to take each photograph manually. In agricultural environments, fixed cameras can support repeated observations and help maintain a consistent image-capture process.

For oyster mushroom cultivation, installing a camera near the cultivation racks can support rack-wise monitoring. The captured images can be processed by a deep learning model, and the results can be associated with the relevant rack records. This approach may reduce the effort required for manual image collection and make it easier to monitor multiple cultivation areas.

The proposed research adopts a fixed-camera approach rather than depending on farmers to upload photographs using their mobile phones. This design is intended to support regular image capture and rack-level monitoring. However, practical implementation must consider camera positioning, image quality, lighting, connectivity, and the ability to identify the correct rack.

2.4.7 Comparison of Existing Approaches with the Proposed System

The following table summarizes the main differences between existing approaches and the proposed research component

Approach	Main Function	Advantages	Limitations
Manual Monitoring	Farmers visually inspect mushroom growth.	Simple, inexpensive, and does not require advanced equipment.	Time-consuming and dependent on human judgment.
Traditional Image Processing	Uses predefined image features and processing rules.	Can be simple and computationally efficient for specific tasks.	May struggle with complex visual variations.
CNN-Based Classification	Classifies images into predefined categories.	Learns visual features from labeled images.	May not provide object locations and requires suitable training data.
YOLO-Based Detection	Detects and locates objects in images.	Supports object localization and efficient image processing.	Performance depends on the dataset, model configuration, and image conditions.
Fixed-Camera Monitoring	Captures images from a consistent location.	Supports repeated observations without	Requires appropriate camera

		manual image uploads.	installation and image quality.
Harvest-Time Prediction	Estimates the remaining time until harvesting.	Can support harvesting schedules and farm planning.	Requires a suitable prediction method and validation against actual harvest records.
Proposed Research Component	Combines YOLOv8-based growth-stage detection with a system designed to support harvest-time estimation and rack-wise monitoring.	Aims to reduce manual monitoring and provide useful harvesting information for Sri Lankan oyster mushroom farmers.	Performance depends on detection reliability, prediction validation, and practical deployment conditions.

Existing mushroom monitoring approaches provide different levels of support, ranging from manual observation to automated detection and growth analysis. Manual methods are straightforward but require continuous human attention, while computer vision and deep learning can automate selected monitoring activities. YOLO-based models provide a practical approach to object detection, and fixed cameras can support repeated image collection in cultivation environments.

However, mushroom detection, growth-stage identification, and harvest-time prediction are distinct tasks that must be evaluated separately. The proposed research focuses on using YOLOv8 to identify four oyster mushroom growth stages and integrating the detection results into a system intended to support rack-wise monitoring and harvest-time estimation. The system uses a fixed-camera approach, a FastAPI backend, MongoDB for record management, and a React Native Android application. Its intended contribution is to make automated mushroom growth monitoring more practical for small and medium-sized oyster mushroom farms in Sri Lanka.

2.5 Gap synthesis and conceptual framework

2.5.1 Gap Synthesis

The literature review identified several approaches to mushroom growth monitoring, including manual observation, traditional image processing, deep learning-based classification, object detection, and harvest-time prediction. These approaches demonstrate the potential of artificial intelligence and computer vision to improve agricultural monitoring. However, several areas require further investigation to develop a practical solution for oyster mushroom farmers.

1. Limitations of Manual Monitoring

Traditional mushroom monitoring depends on farmers regularly observing mushroom bags and identifying their growth conditions. This process can become time-consuming when a farm contains many cultivation bags arranged across multiple racks. Differences in experience and judgment may also affect the consistency of harvesting decisions. Therefore, an automated image-based monitoring approach could reduce the need for repeated manual inspection.

2. Limited Integration of Growth-Stage Detection and Harvest-Time Estimation

Mushroom detection and growth-stage identification are important steps in automated monitoring. However, identifying a mushroom's current growth stage does not automatically determine how much time remains until harvest. A separate prediction method is required to estimate the remaining time. Integrating growth-stage detection with harvest-time estimation could provide more useful information for harvesting decisions.

3. Challenges in Visual Detection

Mushroom appearance can vary according to size, shape, lighting conditions, camera position, and the density of mushroom clusters. These variations may affect the performance of computer vision models. Therefore, a suitable detection model must be trained and evaluated using relevant oyster mushroom images. The proposed research investigates YOLOv8 for identifying four growth-stage classes: early growth, pinning, mature, and ready to harvest.

4. Need for Rack-Level Monitoring

Some monitoring approaches rely on manually captured images or individual observations. These methods may require additional effort when farmers need to monitor several cultivation racks. A fixed-camera approach can support repeated image capture and associate monitoring results with individual racks. However, reliable rack identification, image quality, and consistent camera positioning remain important practical considerations.

5. Need for a Farmer-Friendly System

A technically accurate model is not sufficient if farmers cannot easily understand or use its results. Small and medium-sized oyster mushroom farmers need information that supports practical decisions without requiring advanced technical knowledge. Presenting an estimated time remaining until harvest through a simple application could make the system more useful. The proposed system therefore aims to provide understandable harvesting information through an Android application.

2.5.2 Conceptual Framework

The conceptual framework illustrates how image data and a trained deep learning model can be used to identify oyster mushroom growth stages and support harvest-time estimation. It describes the relationship between the input data, processing methods, system outputs, and intended benefits.

The framework consists of five main components.

1. Input Data

The input consists of oyster mushroom images captured by a fixed camera installed near the cultivation racks. The images contain visible characteristics such as mushroom size, cap shape, color, and cluster development. Appropriate image quality and consistent camera positioning are important for reliable detection.

2. Image Processing and Growth-Stage Detection

The captured images are processed by the trained YOLOv8 model. The model identifies mushrooms and predicts their growth-stage classes. The four classes used in this research are early growth, pinning, mature, and ready to harvest. Detection performance is evaluated using appropriate object-detection metrics.

3. Harvest Time Estimation

The growth-stage detection results are used as inputs to the harvest-time estimation component. A suitable prediction method must estimate the remaining time until harvest. Where available, historical observations and actual harvesting records can support the development and evaluation of this method. The reliability of the estimates must be assessed separately from the performance of the YOLOv8 detection model.

4. System Integration and Information Presentation

The wider system integrates the detection and prediction components with a FastAPI backend, MongoDB database, and React Native Android application. The database can maintain relevant monitoring records, while the application presents the estimated time remaining until harvest and the associated rack information. The interface is intended to be simple enough for farmers to understand without requiring knowledge of deep learning.

5. Intended Outcomes

The expected outcomes include reduced manual monitoring effort, improved consistency in growth observation, better awareness of harvesting readiness, and support for farm-level harvesting decisions. These benefits must be evaluated through model testing and, where possible, feedback from farmers.

2.5.3 Conceptual Framework Diagram

2.5.4 Relationship Between the Framework Components

The conceptual framework assumes that image-based growth-stage detection can provide useful information for harvest-time estimation. The fixed camera captures images from the cultivation environment, and the trained YOLOv8 model analyzes the images to identify the relevant growth stage classes. The resulting information is passed to the harvest-time estimation method, which produces an estimated time remaining until harvest.

The backend manages communication between the model, database, and mobile application. MongoDB stores relevant monitoring records, while the React Native application presents the results to the farmer. Rack identification allows the monitoring information to be associated with the appropriate cultivation area.

The framework also recognizes that several factors may influence system performance. These include image quality, lighting, camera placement, mushroom growth variation, dataset quality, and the reliability of the harvest-time estimation method. Therefore, the detection model and prediction component must be evaluated separately before the system's overall usefulness can be established.

In summary, the conceptual framework provides a structured basis for developing and evaluating an AI based oyster mushroom monitoring system. It connects image capture, deep learning, harvest time estimation, system integration, and farmer-oriented outputs within a single framework.

3 RESEARCH METHODOLOGY

This chapter describes the research methodology used to develop and evaluate the Biological Stage Detection and Harvest Prediction component of the *Rack-Level Micro-Climate & Growth Optimizer for Oyster Mushroom Farms*. The research focuses on using computer vision to identify oyster mushroom growth stages from images captured by a fixed camera positioned near cultivation racks. The component is intended to provide rack-level growth information and support harvesting decisions through estimated time until harvest.

The methodology covers the research design, study materials, experimental procedure, evaluation metrics, data analysis, ethical and security considerations, and reproducibility of the research. It combines the development of an AI-based object detection model with software implementation and performance evaluation.

3.1 Research Design

This research follows an experimental and design-oriented approach. The experimental aspect focuses on training and evaluating a YOLOv8 object detection model for identifying oyster mushroom growth stages. The design-oriented aspect focuses on developing a software component that connects image acquisition, AI-based detection, backend processing, data storage, and a farmer-facing Android application.

The research is organized around three main activities.

1. First, image data representing oyster mushroom growth are collected or obtained from suitable datasets. The images are labelled according to the defined growth-stage classes and prepared for model training and evaluation.
2. Second, the YOLOv8 model is trained to identify the relevant growth-stage classes. The model is evaluated using appropriate object-detection metrics to assess its performance on images that are not used for training.
3. Third, the trained model is integrated into the software component using a FastAPI backend, MongoDB for monitoring records, and a React Native Android application for presenting results. The integration is evaluated through functional tests and, where applicable, performance tests.

The research considers four growth-stage classes: early growth, pinning, mature, and ready-to-harvest. The intended output is practical information about mushroom development and the estimated remaining time until harvest. The user interface is designed to emphasize harvesting information rather than requiring farmers to interpret technical growth-stage labels.

The reported model result is a mean Average Precision at an Intersection over Union threshold of 0.50 (mAP50) of approximately 92.2% after 50 training epochs. This result provides an initial measure of object-detection performance on the evaluation dataset. Further testing is required to determine how well the model performs under different practical cultivation conditions.

Harvest-time prediction is treated as a separate evaluation task. The detection model's mAP50 cannot establish the accuracy of harvest-time estimates. Therefore, if the harvest-time prediction function is implemented, it should be evaluated using actual harvesting observations and suitable prediction-error metrics.

3.2 Data Participants or Study Materials

This research primarily uses image data, AI model resources, software technologies, and mushroom cultivation observations where available. Human participants are relevant only if the study includes farmer usability testing, interviews, questionnaires, or other forms of user evaluation.

3.2.1 Image Dataset

The main study material is a collection of oyster mushroom images representing different stages of development. The images are used to train, validate, and evaluate the YOLOv8 object detection model.

The dataset should represent the four defined classes,

- Early growth
- Pinning
- Mature
- Ready-to-harvest

Each image used for supervised object detection should have appropriate annotations identifying the relevant mushroom objects and their corresponding classes. The quality and consistency of these annotations are important because incorrect labels may affect the model's learning and evaluation results.

Datasets such as the MushR Raw Image Dataset or suitable mushroom growth-stage datasets from platforms such as Kaggle and Roboflow may be considered as data sources. However, the final report should identify the exact datasets used, their sources, the number of images, the annotation format, and any preprocessing performed.

Where images are collected directly from mushroom cultivation racks, the collection process should aim to represent realistic growing conditions. Relevant variations may include mushroom size, cluster arrangement, lighting, camera distance, and visible growth characteristics.

3.2.2 Development and Evaluation Materials

The technical materials used in the research include the trained YOLOv8 model, Python-based development tools, the FastAPI backend, MongoDB database, and React Native Android application.

The trained model weights and configuration files are required for model inference. The backend is used to receive image-processing requests and return detection results. MongoDB stores relevant monitoring records, while the Android application presents the available information to users.

A fixed camera positioned near cultivation racks is the intended image acquisition method. This approach differs from a workflow in which farmers manually upload photographs using their mobile phones.

3.2.3 Human Participants

If farmer usability testing is conducted, the participants should represent the intended users of the system, particularly small- and medium-scale oyster mushroom farmers. Their feedback may be used to evaluate whether the monitoring information is understandable and useful for cultivation decisions.

If the study includes ten farmer participants, the report should state this number only after those participants have actually completed the evaluation. Their selection method, relevant characteristics, testing activities, and feedback collection method should also be documented.

If no human-participant study has been conducted, the research should be described as a technical evaluation based on image data and software tests rather than a completed farmer usability study.

3.3 Experimental or Study Procedure

The research procedure consists of a series of stages, beginning with dataset preparation and continuing through model training, software integration, and evaluation.

3.3.1 Dataset Preparation

The first stage involves collecting or obtaining images that represent oyster mushroom growth. The images are reviewed to determine whether they are suitable for training and evaluation.

Each relevant mushroom object is labelled according to its growth stage. The annotation process should follow consistent rules so that similar mushroom appearances are assigned to the same class when appropriate.

After annotation, the dataset is divided into training, validation, and testing subsets. The training subset is used to teach the model, the validation subset supports model monitoring and configuration decisions, and the testing subset provides an independent evaluation of the selected model.

Where multiple images originate from the same mushroom cluster, rack, or cultivation session, the data-splitting process should avoid placing closely related observations in different subsets where this could cause data leakage. This helps produce a more realistic estimate of model performance.

The dataset size, class distribution, split proportions, annotation tools, and preprocessing settings should be recorded as part of the research documentation.

3.3.2 YOLOv8 Model Training

The prepared dataset is used to train the YOLOv8 object detection model. The model is configured to identify the four defined oyster mushroom growth-stage classes.

Training involves supplying the annotated images to the model and updating its parameters to improve object detection. The training configuration should record important settings, including the number of epochs, image size, batch size, optimizer settings where applicable, and confidence-related settings used during inference.

The model was trained for 50 epochs and achieved a reported mAP50 of approximately 92.2% on the evaluation dataset. The final report should preserve the actual evaluation output and clarify whether the reported value was obtained from the validation or independent test subset.

The selected model weights are retained for use during inference and software integration. The configuration and class definitions should be maintained to ensure that the deployed model produces results consistent with the training setup.

3.3.3 Model Evaluation

After training, the selected model is evaluated using images that were not used to update its parameters. The evaluation examines its ability to identify mushroom objects and assign them to the correct growth-stage classes.

The analysis should include mAP50 and, where available, precision, recall, class-specific performance, and a confusion matrix or other suitable class-level error analysis. These measures help identify whether some growth stages are more difficult to detect than others.

The evaluation should also consider practical image conditions, such as lighting differences, overlapping mushroom clusters, and changes in mushroom appearance. These factors may affect the reliability of the model when it is used in a real cultivation environment.

3.3.4 Backend and Database Integration

The trained model is integrated into the FastAPI backend. The backend receives an image and its associated rack information, processes the request, invokes the model, and returns the detection results.

Where implemented, the relevant monitoring records are stored in MongoDB. These records may include the rack identifier, observation timestamp, detected classes, confidence scores, image reference, model version, and harvest-time estimate.

The integration process should verify that images are processed correctly and that results are associated with the correct rack and observation time. Errors such as invalid image input, failed inference, and database connection problems should also be tested.

3.3.5 Android Application Integration

The React Native Android application communicates with the backend to retrieve and display available monitoring information.

The interface is designed to help farmers review rack-level monitoring results without needing to understand the internal operation of the AI model. Where the harvest-time prediction function is available, the application should emphasize the estimated remaining time until harvest.

Application testing should confirm that the interface displays valid results, handles unavailable information appropriately, and communicates backend errors clearly.

3.3.6 Harvest-Time Prediction Evaluation

Harvest-time prediction is evaluated separately from growth-stage detection. If the implemented method estimates the remaining time until harvest, its predictions should be compared with actual observed harvesting times.

For each evaluated observation, the predicted remaining time is compared with the actual remaining time recorded for the relevant mushrooms or cultivation batch. Suitable metrics, such as mean absolute error, may then be calculated.

The evaluation should document the prediction method, the source and quantity of the supporting data, the definition of the prediction target, and the method used to establish actual harvesting times.

If reliable historical observations or actual harvesting records are unavailable, the research should not claim that harvest-time prediction accuracy has been experimentally established. In that situation, the function should be described as a proposed or preliminary capability requiring further validation.

3.4 Measures and Evaluation Metrics

The research uses different measures to evaluate the AI model, software component, and harvest-time estimation function. Each metric addresses a different aspect of system performance.

3.4.1 Object-Detection Metrics

Precision: Precision measures the proportion of positive detections that are correct. Higher precision indicates fewer false-positive detections.

Recall: Recall measures the proportion of actual objects that the model successfully detects.

Intersection over Union (IoU): IoU measures the overlap between a predicted bounding box and the corresponding ground-truth bounding box.

Mean Average Precision at IoU 0.50 (mAP50): mAP50 measures the mean Average Precision across the defined classes using an IoU threshold of 0.50. The reported value of approximately 92.2% is the current object-detection result for this research component.

These metrics should be interpreted together where the required evaluation results are available. A single mAP50 value does not fully describe performance for every class or under every cultivation condition.

3.4.2 Harvest-Time Prediction Metrics

If the harvest-time prediction function is implemented and actual harvesting records are available, the following metrics can be used.

Mean Absolute Error (MAE): MAE measures the average absolute difference between predicted and actual remaining time until harvest.

The unit of MAE should match the unit used for the prediction target, such as hours or days.

Root Mean Squared Error (RMSE): RMSE measures prediction error while giving greater weight to larger errors.

MAE and RMSE should only be reported when predictions can be compared with reliable actual harvesting-time observations. They should not be calculated from the model's mAP50 result.

3.4.3 Software Performance Metrics

3.4.3 Software Performance Metrics

The software component should be evaluated using functional and performance-related measures.

Relevant measures include:

- **Image-processing response time:** The time required to process an image and return the detection result.
- **Request success rate:** The proportion of valid processing requests that complete successfully.
- **Data-storage correctness:** Whether monitoring records are stored with the correct rack identifiers, timestamps, and detection information.
- **Integration correctness:** Whether the backend, model, database, and Android application exchange information correctly.
- **Error-handling performance:** Whether invalid images, unavailable services, and failed requests are handled appropriately.

The project has identified an image-processing target of less than three seconds. This should be treated as a target until verified through testing. Any reported response time should state the test environment, number of requests, and measurement method.

3.4.4 Usability Measures

If farmer usability testing is conducted, the system may be evaluated using feedback about ease of use, clarity of monitoring information, usefulness of harvest-time estimates, and confidence in the displayed results.

Questionnaires, short interviews, task-completion observations, or structured feedback forms may be used to collect this information. The report should state the number of participants, questions or measures used, and the method of analysis.

If usability testing has not been conducted, usability should be described as a design objective rather than a proven outcome.

3.5 Data Analysis

The data analysis process focuses on interpreting the model evaluation results, examining software functionality, and determining the reliability of harvest-time estimates where sufficient data are available.

3.5.1 Analysis of Object-Detection Results

The YOLOv8 evaluation results are analyzed using the available precision, recall, mAP50, and class-level evaluation outputs. The reported mAP50 of approximately 92.2% is used as an initial indicator of detection performance on the evaluation dataset.

Where class-level results are available, the performance of early growth, pinning, mature, and ready-to-harvest detection is compared. This analysis can help identify classes that are frequently confused and may require additional training data or improved annotations.

False-positive and false-negative detections should also be examined where possible. Reviewing incorrect predictions may help identify the influence of lighting, overlapping mushrooms, insufficient image quality, or similarities between growth stages.

The findings should be interpreted according to the dataset and evaluation conditions. The results should not be generalized to all oyster mushroom farms without additional testing under different practical conditions.

3.5.2 Analysis of Harvest-Time Estimates

If harvest-time predictions and actual harvesting records are available, the predicted and actual values are compared using MAE, RMSE, or other suitable measures.

The analysis should examine the size of prediction errors and identify situations in which the estimates are less reliable. For example, differences in growth rates or insufficient observations may contribute to inaccurate estimates.

If the data are insufficient to calculate reliable prediction metrics, the report should explain this limitation and avoid presenting unsupported accuracy claims.

3.5.3 Analysis of Software Tests

The results of functional and integration tests are analyzed to determine whether the component satisfies its specified requirements. Test cases should cover image submission, model inference, result processing, database storage, application display, and error handling.

For each test, the expected result is compared with the observed result. Failed tests should be documented, investigated, and repeated after corrective changes where appropriate.

Response-time measurements should be summarized using suitable statistics, such as the mean and maximum observed processing time. The number of tests and the computing environment should be recorded to support interpretation.

3.5.4 Interpretation of Overall Findings

The overall findings are determined by considering AI detection performance, software functionality, response time, and harvest-time prediction results where available.

A successful object-detection evaluation does not automatically establish that the entire application is reliable or that harvest-time estimates are accurate. Each aspect should therefore be evaluated separately before conclusions are drawn about the effectiveness of the complete component.

3.6 Ethics, Privacy, Security and Responsible AI

The research follows responsible practices in data collection, software development, model evaluation, and reporting. Although the main input consists of oyster mushroom images, camera-based monitoring and stored records should still be managed carefully.

3.6.1 Ethical Data Use

Images used for training and evaluation should be obtained from sources that permit the intended research use. Dataset licences, usage restrictions, and attribution requirements should be checked before images are included in the project.

If images are collected directly from farms, permission should be obtained from the relevant farm owner or responsible person. Any human-participant activities, such as

farmer interviews or usability tests, should follow the university's applicable research ethics requirements.

3.6.2 Privacy Protection

The fixed camera should be positioned to focus on mushroom cultivation areas. Unnecessary recording of people or unrelated areas should be avoided.

If personal information is captured, suitable privacy measures should be applied. Access to stored images and monitoring records should be limited to authorized users according to the needs of the research.

3.6.3 System Security

The backend and database should be configured to reduce unauthorized access and accidental data loss. Appropriate security measures may include access controls, protected credentials, secure communication, input validation, and database backup procedures.

Sensitive configuration information should not be included in public source-code repositories. Model files, source code, and deployment settings should be stored and managed appropriately.

Security measures should be described as implemented only when they have actually been configured and tested.

3.6.4 Responsible AI and Prediction Limitations

The YOLOv8 model may produce incorrect detections because of lighting variations, overlapping mushroom clusters, image quality, or differences in mushroom appearance. These limitations should be considered when presenting the results.

Harvest-time estimates should be treated as decision-support information rather than guaranteed harvesting dates. Farmers should retain the ability to inspect mushrooms directly and use their cultivation knowledge when making final harvesting decisions.

The system should avoid displaying misleading estimates when the model output is unreliable or required information is unavailable. Uncertainty and known limitations should be documented in the research report.

3.6.5 Research Integrity

All reported results should be based on actual experiments, measurements, or collected feedback. The reported mAP50 of approximately 92.2% should be supported by the model evaluation output.

Performance targets, planned features, and expected outcomes should be clearly distinguished from verified achievements. Research sources, datasets, and software tools should be acknowledged according to the university's required referencing style.

3.7 Reproducibility and Research Artefacts

Reproducibility is important because it allows the research methods and reported results to be reviewed and, where possible, repeated by other researchers. The project should maintain the technical information and artefacts needed to understand how the model was trained, evaluated, and integrated into the software component.

3.7.1 Dataset Documentation

The dataset documentation should identify the image sources, dataset version where available, annotation method, class definitions, and preprocessing steps. The training, validation, and testing split should also be documented.

Where dataset licences permit, suitable dataset references or access instructions should be included. If images cannot be redistributed, the report should explain the applicable restrictions.

3.7.2 Model and Training Artefacts

The main model-related artefacts include:

- Trained YOLOv8 model weights.
- Model configuration and class definitions.
- Training and validation settings.
- Training logs and available evaluation results.
- Precision, recall, mAP50, and other calculated metrics where available.
- Software and library versions required to run the model.

These artefacts help document the relationship between the training procedure and the reported detection results.

3.7.3 Software Artefacts

The software artefacts include the FastAPI backend source code, React Native Android application source code, database structure, API definitions, and relevant configuration files.

Where Node-RED and MQTT are implemented, the relevant flow configurations and communication settings should also be documented. Secrets, passwords, and other sensitive configuration values should be excluded from shared artefacts.

3.7.4 Testing Artefacts

The testing documentation should contain test cases, expected outcomes, observed outcomes, and relevant evaluation evidence. Performance measurements should include sufficient information about the computing environment and test conditions to allow the results to be interpreted.

If farmer usability testing is conducted, the research should retain the relevant questionnaire or evaluation form and an appropriately anonymized summary of the results, subject to ethical and privacy requirements.

3.7.5 Version Control and Reproducible Execution

Version control should be used where available to track changes to the source code and configuration. Important model versions should be identified so that the model used during evaluation can be distinguished from later versions.

A requirements file or equivalent environment specification should record the relevant Python libraries and software dependencies. Instructions for configuring the backend, loading the trained model, connecting to MongoDB, and running the Android application should be maintained.

The report should document the actual versions, commands, and settings used during the experiments rather than relying on general instructions.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements of the Biological Stage Detection and Harvest Prediction component were identified to support the automatic monitoring of oyster mushroom growth in cultivation racks. The main purpose of this component is to reduce the need for frequent manual observation by providing image-based growth monitoring and estimated harvesting time. The system combines a fixed-camera image acquisition setup, a YOLOv8-based object detection model, a FastAPI backend, a MongoDB database, and a React Native Android application.

The requirements are divided into functional requirements, which describe the tasks the system should perform, and non-functional requirements, which describe the expected quality, performance, usability, and reliability of the system.

4.1.1 Functional Requirements

The functional requirements describe the main operations of the component.

- FR1 – Image Acquisition: The system should obtain images of oyster mushrooms using a fixed camera positioned near the cultivation racks. Images should be associated with the relevant rack identifier and observation timestamp.
- FR2 – Growth-Stage Detection: The system should process captured images using the trained YOLOv8 model to identify oyster mushroom growth stages. The defined classes are early growth, pinning, mature, and ready-to-harvest.
- FR3 – Detection Result Processing: The backend should receive the model's output and process the detected classes and confidence scores before making the results available to the application.
- FR4 – Rack-Level Monitoring: The system should associate monitoring results with the correct rack so that farmers can review growth information for individual cultivation racks.
- FR5 – Harvest-Time Estimation: Where implemented, the system should provide an estimate of the remaining time until the mushrooms are ready to harvest. The estimate should support harvesting decisions rather than guarantee an exact harvesting date.

- FR6 – Data Storage: The system should store relevant monitoring records in MongoDB, including rack identifiers, observation timestamps, detection results, and harvest-time estimates where available.
- FR7 – Information Presentation: The React Native Android application should present the available monitoring information in a simple and understandable format. The interface should emphasize the estimated time until harvest rather than requiring farmers to interpret technical growth-stage labels.
- FR8 – Backend Communication: The FastAPI backend should receive image-processing requests, communicate with the detection model, and return the available results to the Android application.
- FR9 – Error Handling: The system should handle invalid images, unsuccessful processing requests, and communication failures by providing appropriate error responses or status messages.

4.1.2 Non-Functional Requirements

The non-functional requirements define the expected quality and operational characteristics of the component.

- NFR1 – Detection Performance: The YOLOv8 model should identify the defined growth-stage classes with an acceptable level of accuracy. The model achieved a reported mAP50 of approximately 92.2% during evaluation. This result should be interpreted according to the evaluation dataset and should not be treated as a guarantee of the same performance under all farm conditions.
- NFR2 – Response Time: The image-processing workflow should return results within a practical time for farm monitoring. The project target is to process an image in less than three seconds under the specified test conditions. This target should be verified through performance testing.
- NFR3 – Usability: The Android application should be easy for small- and medium-scale oyster mushroom farmers to understand and use. Important information should be presented clearly, with minimal need for technical knowledge.

- NFR4 – Reliability: The system should process valid images consistently and associate detection results with the correct rack and timestamp. Errors in image processing or communication should be handled appropriately.
- NFR5 – Maintainability: The system should use a modular architecture that separates image acquisition, AI processing, backend services, database storage, and the user interface. This structure should make it easier to test, maintain, and update individual components.
- NFR6 – Scalability: The system architecture should allow additional cultivation racks and monitoring records to be supported as the farm expands, subject to available computing, storage, and network resources.
- NFR7 – Data Integrity: Monitoring records should be stored with the correct rack identifiers and timestamps. Suitable validation should be applied to reduce incomplete, duplicated, or incorrectly associated records.
- NFR8 – Security: Access to backend services and stored monitoring records should be controlled through appropriate authentication and authorization mechanisms where implemented. Communication and data storage should be configured to reduce the risk of unauthorized access.
- NFR9 – Compatibility: The Android application, FastAPI backend, YOLOv8 model, and MongoDB database should operate together using compatible interfaces and data formats.
- NFR10 – Environmental Robustness: The component should be evaluated under different practical conditions, including variations in lighting, mushroom size, cluster overlap, and camera position, to understand how these factors affect detection performance.

4.2 Component architecture within the complete solution

The Biological Stage Detection and Harvest Prediction component is part of the Rack-Level Micro-Climate & Growth Optimizer for Oyster Mushroom Farms. The complete system is designed to support oyster mushroom farmers by combining image-based growth monitoring with environmental monitoring to improve cultivation management and harvesting decisions.

The main responsibility of this component is to analyze images captured by a fixed camera positioned near the oyster mushroom cultivation racks. The trained YOLOv8 model identifies the visible growth-stage classes, while the system uses the available detection information to support harvest-time estimation. The component also stores monitoring records and presents useful information through a React Native Android application.

4.2.1 Architectural Overview

The component follows a modular architecture consisting of five main layers: image acquisition, AI processing, backend services, data storage, and user interaction. Each layer performs a specific task and communicates with the other layers through defined interfaces.

1. Image Acquisition Layer

A fixed camera is positioned near the oyster mushroom cultivation racks to capture images of mushroom development. Each observation should be associated with the relevant rack identifier and timestamp. This enables the system to maintain separate monitoring records for different racks.

2. AI Processing Layer

The AI processing layer uses the trained YOLOv8 model to analyze captured images. The model identifies four defined classes: early growth, pinning, mature, and ready-to-harvest. It returns detection results and confidence scores for further processing by the backend.

3. Backend Service Layer

The FastAPI backend manages communication between the image-processing functionality, database, and Android application. It receives image-processing requests, executes model inference, processes the results, and returns the available monitoring information to the application.

4. Data Storage Layer

MongoDB is used to store relevant monitoring records, including rack identifiers, timestamps, detection results, and harvest-time estimates where available. These records support the review of mushroom development over time.

5. Application Layer

The React Native Android application provides the user interface for farmers. It is designed to present rack-level monitoring information in a simple and understandable format. The interface emphasizes the estimated remaining time until harvest rather than requiring farmers to interpret technical AI model outputs.

4.2.2 Interaction With the Complete System

The Biological Stage Detection and Harvest Prediction component operates alongside the microclimate monitoring component. The microclimate component collects environmental measurements, such as temperature, humidity, and other relevant environmental variables. In contrast, the biological detection component analyzes visual images to identify mushroom development.

The two components provide complementary information about cultivation conditions. Environmental data describe the conditions surrounding the mushrooms, while image-based results indicate their visible development. When both sets of records are associated with the same rack and relevant observation times, they can provide a more complete overview of the cultivation process.

Node-RED and MQTT may be used to support communication between system components where these technologies are included in the implemented integration. However, environmental measurements should only be used directly in the harvest-time prediction process if the prediction method has been designed and implemented to use them.

4.2.3 Data Flow Between Components

The data flow of the Biological Stage Detection and Harvest Prediction component follows these steps,

- Image capture: The fixed camera captures images of oyster mushrooms growing on the cultivation racks.
- Image transmission: The captured images are submitted to the FastAPI backend for processing.
- Growth-stage detection: The trained YOLOv8 model analyzes the images and identifies the relevant growth-stage classes.

- Result processing: The backend receives the detection results and associates them with the relevant rack identifier and timestamp.
- Data storage: The relevant monitoring information is stored in MongoDB where the storage functionality is implemented.
- Harvest-time estimation: The system provides an estimate of the remaining time until harvest if the corresponding prediction functionality is available.
- Information presentation: The Android application displays the available monitoring information to the farmer.

This sequence enables image-based observations to be transformed into information that can support cultivation monitoring and harvesting decisions.

4.2.4 Architectural Design Considerations

The architecture is designed with modularity, maintainability, and usability in mind. Separating image acquisition, AI processing, backend services, data storage, and the application interface makes it easier to test and maintain individual parts of the system.

The use of rack identifiers and timestamps supports organized monitoring across multiple cultivation racks. The backend provides a common interface for processing requests and returning results, while MongoDB supports the storage of monitoring records.

The architecture also considers practical operating conditions. Image quality, lighting, network availability, and computing resources can affect system performance. Therefore, the component should be tested under realistic cultivation conditions before being relied upon for routine farm operations.

4.3 Design alternatives and decisions

During the design of the Biological Stage Detection and Harvest Prediction component, several alternatives were considered for image acquisition, mushroom growth-stage detection, backend development, data storage, and information presentation. The main objective was to select an approach that could identify oyster mushroom growth stages, support harvesting decisions, and provide useful information to small- and medium-scale farmers.

The design decisions considered the requirements of the research, the available development resources, the need for future integration with other system components, and the usability of the final solution.

4.3.1 Image Acquisition Approach

Two main approaches can be considered for collecting mushroom images: manual image capture using a mobile phone and automatic image acquisition using a fixed camera.

Manual image capture is simple and inexpensive because farmers can take photographs whenever they want to check mushroom growth. However, it requires farmers to capture images regularly. The images may also differ in angle, distance, and lighting, which can affect detection consistency.

The fixed-camera approach was selected as the intended design because it supports repeated image capture from a consistent position near the cultivation racks. This approach is suitable for rack-level monitoring and reduces the need for farmers to take photographs manually. However, camera placement, lighting, and image quality must be managed carefully.

4.3.2 Growth-Stage Detection Approach

Several image-processing approaches can be used to identify mushroom growth stages. Traditional image-processing techniques rely on manually defined features, such as color, shape, and size. Although these methods can be computationally simple, their performance may decrease when lighting conditions and mushroom appearances vary.

A conventional image-classification model can assign a growth-stage label to an image. However, object detection can also identify and locate relevant mushroom objects within an image, which is useful when multiple mushrooms or clusters appear in the same frame.

YOLO-based object detection was selected for this component because it can identify objects and their locations in a single inference process. The trained YOLOv8 model achieved a reported mAP50 of approximately 92.2% after 50 training epochs on the evaluation dataset.

4.3.3 Backend Technology

Two possible backend approaches are a Node.js-based service and a Python-based service. Node.js is suitable for handling application requests and general web services. However, the AI model used in this component is based on YOLOv8, which operates within the Python machine-learning ecosystem.

FastAPI was selected as the backend framework because it supports Python-based model integration and provides an interface for receiving image-processing requests and returning detection results. It also allows the backend to communicate with the database and Android application.

4.3.4 Database Selection

A relational database and a NoSQL database are two possible options for storing monitoring information. Relational databases organize information into tables with predefined relationships. They are useful when data structures and relationships are relatively fixed.

MongoDB was selected for this component because it stores records in a document-oriented format. This provides flexibility when storing image-detection results, rack identifiers, timestamps, confidence scores, and harvest-time estimates. It also supports the storage of monitoring records with different sets of attributes when required.

4.3.5 Mobile Application Development

A web application and a mobile application are possible approaches for presenting monitoring information to farmers. A web application can be accessed through a browser, while a mobile application can provide a dedicated interface for Android devices.

React Native was selected for developing the Android application because it supports mobile user-interface development using JavaScript and React-based development practices. It can communicate with the FastAPI backend to retrieve and display monitoring results.

The interface is intended to present practical information, particularly the estimated remaining time until harvest, in a simple format. This reduces the need for farmers to understand technical AI outputs.

4.3.6 Harvest-Time Estimation Approach

Harvest-time estimation can be approached using growth-stage information, historical observations, environmental measurements, or a combination of these data sources. A simple approach could assign an estimated remaining time to each detected growth stage. However, this may not account for differences in mushroom growth rates.

A more advanced prediction method could use historical growth records and environmental measurements to estimate the remaining time until harvest. Such an approach would require suitable training data and separate validation to establish its reliability.

For this component, harvest-time estimation is intended to help farmers understand when mushrooms may be ready for harvesting. The selected prediction method should depend on the available data and the functionality implemented in the system.

4.3.7 System Integration Approach

A tightly coupled architecture could combine all system functions into a single application. However, this could make individual functions more difficult to maintain or update independently. A modular architecture separates image acquisition, AI processing, backend services, data storage, and the user interface.

The modular approach was selected because it supports independent development and testing of individual functions. It also provides a foundation for integration with the microclimate monitoring component. Node-RED and MQTT may support communication between components where they are included in the implemented architecture.

4.4 Detailed design

The detailed design of the Biological Stage Detection and Harvest Prediction component describes how the selected technologies and system components work together to monitor oyster mushroom growth. The design is based on a fixed-camera image acquisition setup, a trained YOLOv8 object detection model, a FastAPI backend, a MongoDB database, and a React Native Android application.

The main objective is to transform captured mushroom images into useful monitoring information that can help farmers understand mushroom development and make harvesting decisions. The design also supports rack-level record management and future integration with the microclimate monitoring component.

4.4.1 Overall System Design

The component is designed using a modular architecture. Each module has a specific responsibility, and the modules communicate through defined interfaces.

The main modules are:

1. Image Acquisition Module: Captures images of oyster mushrooms using a fixed camera positioned near the cultivation racks.
2. Image Processing Module: Prepares captured images for processing by the trained YOLOv8 model.
3. Growth-Stage Detection Module: Identifies mushroom growth-stage classes and produces detection results.
4. Backend Processing Module: Uses FastAPI to manage image-processing requests and return results.
5. Data Management Module: Uses MongoDB to store rack identifiers, timestamps, detection results, and harvest-time estimates where available.
6. Harvest-Time Estimation Module: Provides an estimate of the remaining time until harvest using the implemented prediction method.
7. User Interface Module: Uses a React Native Android application to display monitoring information to farmers.

The modular design supports independent development and testing of the individual modules while allowing them to operate as part of the complete component.

4.4.2 Image Acquisition and Preprocessing

The image acquisition module is responsible for obtaining visual information about oyster mushroom development. A fixed camera is positioned near the cultivation racks to capture images from a consistent location.Each captured image should be associated with a rack identifier and observation timestamp. This information is important for maintaining separate monitoring records for different racks and reviewing growth over time.

Before model inference, the image is prepared according to the preprocessing requirements used during YOLOv8 training. These steps may include image resizing, format conversion, and normalization where required by the model implementation. Using compatible preprocessing settings helps maintain consistency between training and inference.

Image quality is an important consideration because blurred images, poor lighting, overlapping mushroom clusters, and unsuitable camera positioning may affect detection performance.

4.4.3 YOLOv8 Growth-Stage Detection Design

The growth-stage detection module uses the trained YOLOv8 model to identify oyster mushrooms and their associated growth-stage classes. The model was trained for 50 epochs and achieved a reported mAP50 of approximately 92.2% on the evaluation dataset.

The model is designed to recognize four classes:

1. Early growth
2. Pinning
3. Mature
4. Ready-to-harvest

When an image is submitted for processing, the model analyzes the image and produces detection results, including the predicted class, object location, and confidence score where supported by the model configuration.

The backend receives these results and prepares the relevant information for storage and presentation. Confidence scores can also be used to identify detections that may require additional review, depending on the implemented decision rules.

The reported mAP50 indicates object-detection performance on the evaluation dataset. It does not independently demonstrate that every growth-stage class has the same accuracy or that the harvest-time estimates are equally reliable.

4.4.4 Backend Processing Design

The backend is developed using FastAPI and Python. It provides the communication interface between the AI model, database, and Android application.

The backend is responsible for receiving image-processing requests, validating incoming data, invoking the YOLOv8 model, processing the model output, and returning the available results.

A typical backend workflow consists of the following steps,

1. Receive an image-processing request.
2. Validate the image and associated rack information.
3. Prepare the image for model inference.
4. Execute the YOLOv8 model.
5. Process the detection results and confidence scores.
6. Generate or retrieve the harvest-time estimate if the prediction function is implemented.
7. Store the relevant monitoring record in MongoDB.
8. Return the available results to the Android application.

Appropriate error handling should be provided for invalid images, unsuccessful inference, database connection failures, and communication errors. This helps prevent individual failures from causing unexpected application behavior.

4.4.5 Database Design

MongoDB is used to store the monitoring information generated by the component. A document-oriented structure provides flexibility for storing detection results and associated monitoring information.

A monitoring record may contain the following fields,

Field	**Description**
Record ID	Unique identifier for the monitoring record.
Rack ID	Identifier of the cultivation rack.
Observation Timestamp	Date and time of image capture or observation.

Image Reference	Reference to the captured image, where images are retained.
Detected Classes	Growth-stage classes identified by the model.
Confidence Scores	Confidence values associated with the detected classes.
Harvest-Time Estimate	Estimated remaining time until harvest, if available.
Model Version	Version of the model used to generate the result, where recorded.

The database structure supports the organization of monitoring records by rack and observation time. It can also help compare successive observations to review changes in mushroom development.

Large image files may be stored separately, with MongoDB maintaining references to them. This approach can reduce the need to store all image data directly inside monitoring documents.

The final database structure should reflect the fields actually implemented in the system.

4.4.6 Harvest-Time Estimation Design

The harvest-time estimation function is intended to provide farmers with an estimate of how much time remains before oyster mushrooms are ready for harvesting. It is separate from the YOLOv8 detection function because identifying a growth stage does not, by itself, guarantee an accurate prediction of harvesting time.

The prediction method should use the information supported by the implemented design. For example, a stage-based approach may associate detected growth stages with estimated remaining time ranges. A data-driven approach may use historical growth observations and, where supported, environmental measurements to estimate the remaining time.The estimated time should be presented as an approximation rather than a guaranteed harvesting date. Actual mushroom growth may vary according to cultivation conditions, strain, substrate, and other factors.

The Android interface is intended to emphasize the remaining time until harvest rather than displaying technical growth-stage labels as the main output. The prediction method should be evaluated separately using suitable measurements, such as the mean absolute error (MAE), when reliable actual harvesting-time records are available.

If the harvest-time estimation function has not yet been implemented, this module should be treated as a planned part of the design rather than a completed system feature.

4.4.7 Android Application Design

The user interface is developed using React Native for Android devices. Its main purpose is to present monitoring information in a simple format that small- and medium-scale oyster mushroom farmers can understand.

The interface is intended to support the following functions,

1. Viewing monitoring information for individual cultivation racks.
2. Viewing the latest available detection results.
3. Checking the estimated remaining time until harvest, where available.
4. Reviewing observation times and historical monitoring records, where implemented.
5. Identifying unavailable or unsuccessful monitoring results through appropriate status messages.

The interface should avoid unnecessary technical information and prioritize details that support practical cultivation decisions. For example, estimated harvesting time can be displayed prominently for each rack when a valid estimate is available.

The final interface should reflect the functionality implemented in the Android application.

4.4.8 Integration Design

The component is designed to operate alongside the microclimate monitoring component within the complete oyster mushroom monitoring solution.

The biological detection component provides image-based growth information, while the microclimate component collects environmental measurements. Rack identifiers and timestamps can be used to associate observations from both components.

FastAPI provides the backend interface for the biological detection functionality, while MongoDB can store its monitoring records. Node-RED and MQTT may support communication with other components where these technologies are included in the implemented architecture.

Although the two components provide complementary information, environmental measurements should only be incorporated directly into harvest-time prediction if the prediction method has been developed to use those measurements. Otherwise, the components should be described as providing separate but related monitoring information.

4.5 Standards and professional practice

The development of the Biological Stage Detection and Harvest Prediction component follows appropriate software engineering practices, data management principles, and responsible artificial intelligence practices. These practices aim to improve the reliability, maintainability, usability, and safety of the system. Since the component is intended to support oyster mushroom farmers, particular attention is given to detection reliability, data accuracy, user privacy, and the responsible presentation of harvest-time estimates.

4.5.1 Software Engineering Standards and Practices

The component follows a modular software design approach by separating image acquisition, AI processing, backend services, database management, and the mobile application interface. This separation supports independent development, testing, and maintenance of individual modules.

The backend is developed using FastAPI, while the Android application is developed using React Native. Clear interfaces between these modules help maintain consistency when exchanging image-processing requests and detection results. Appropriate naming conventions, code documentation, input validation, error handling, and version control should be used to improve code quality and maintainability.

Software testing should be performed at different levels, including unit testing of individual functions, integration testing between modules, and system testing of the

complete workflow. Changes to the model, backend, database, or application should be tested to identify possible errors before being included in the working system.

These practices are consistent with the general software quality principles described in ISO/IEC 25010, which provides a model for evaluating software product quality.

4.5.2 Artificial Intelligence and Model Evaluation

The YOLOv8 model is used to identify oyster mushroom growth-stage classes from captured images. Professional practice requires the model to be evaluated using appropriate data and performance measures rather than relying only on its training performance.

The dataset should contain correctly labelled images representing the defined growth stages. Training, validation, and testing data should be separated appropriately to reduce the risk of overly optimistic evaluation results. Where possible, images of the same mushroom cluster or closely related observations should not be distributed across different data splits in a way that causes data leakage.

The model achieved a reported mAP50 of approximately 92.2% after 50 training epochs. This result should be reported with the evaluation conditions and recognized as a measure of object-detection performance on the evaluated dataset. It should not be interpreted as a guarantee of the same performance in every cultivation environment.

The model should also be evaluated under practical conditions, including changes in lighting, camera position, mushroom appearance, and overlapping clusters. Harvest-time prediction should be evaluated separately because object-detection metrics do not measure the accuracy of estimated harvesting times.

4.5.3 Data Management and Privacy

The component may process images of oyster mushrooms and store monitoring records containing rack identifiers, timestamps, detection results, and harvest-time estimates. Appropriate data management practices should be followed to maintain data integrity, availability, and confidentiality.

Only information necessary for the research and system operation should be collected. Access to the backend, database, and monitoring records should be restricted

according to the needs of the application. If images or records are shared for research, testing, or model improvement, appropriate authorization should be obtained where required.

Backups should be maintained where the system stores important monitoring records, and recovery procedures should be tested when backup functionality is implemented. Model files, source code, and configuration settings should also be managed carefully to reduce the risk of accidental loss.

If the camera captures identifiable individuals or other personal information, additional privacy measures and appropriate consent may be required. Camera placement should therefore focus on mushroom cultivation areas rather than unnecessary recording of people.

4.5.4 Responsible and Ethical Use of Artificial Intelligence

The system is intended to support farmers in monitoring oyster mushroom growth and making harvesting decisions. It should therefore be developed and presented as a decision-support tool rather than as a replacement for all human observation and agricultural knowledge.

Harvest-time estimates should be presented as approximations because actual growth rates can vary according to cultivation conditions and other biological factors. The interface should avoid presenting predictions as guaranteed harvesting dates.

The system should also communicate uncertainty appropriately. When an image is unclear, a detection is unreliable, or a valid estimate is unavailable, the application should avoid displaying misleading information. Suitable status messages or instructions to check the mushrooms manually can help reduce the risk of decisions based on unreliable outputs.

Testing should consider different mushroom appearances and practical cultivation environments to identify situations in which the model may perform less reliably. These limitations should be documented honestly in the research report.

4.5.5 Professional Documentation and Research Integrity

Professional research practice requires accurate reporting of the methods, implementation, experiments, and results. The report should distinguish between completed functionality, ongoing development, and planned improvements.

The reported mAP50 of approximately 92.2% should be supported by the relevant model evaluation results. Other performance claims, such as image-processing time, harvest-time prediction accuracy, and farmer satisfaction, should only be reported as achieved when supported by actual measurements or collected evidence.

Technical documentation should describe the system architecture, database structure, model configuration, API interfaces, and testing procedures sufficiently clearly to support maintenance and reproducibility. External research, datasets, software libraries, and technical resources should be acknowledged using the referencing style required by the university.

Version control and development records should be maintained where available to track changes and support collaboration with other members of the research project.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The development environment for the Biological Stage Detection and Harvest Prediction component was selected to support deep learning-based image analysis, backend development, database management, and mobile application development. The system focuses on identifying oyster mushroom growth stages from images and supporting the estimation of the remaining time until harvest. The selected technologies provide the foundation for integrating the trained deep learning model with the wider monitoring application.

5.1.1 Development Environment

The development environment consists of software tools and technologies used to develop, train, test, and integrate the system components. Python is used for deep learning development and backend implementation, while React Native supports the development of the Android application. MongoDB is used to manage relevant monitoring records and prediction information.

The YOLOv8 model is developed and trained using labeled oyster mushroom images. The trained model is then integrated into the backend to process input images and return growth-stage detection results. The backend communicates with the database and provides the necessary information to the mobile application.

The development environment includes the following tools:

5.1.2 Technology Stack

The technology stack is organized into four main layers: image acquisition, AI processing, backend and data management, and user interaction.

1. Image Acquisition Layer

The image acquisition layer uses a fixed camera installed near the oyster mushroom cultivation racks. The camera captures images of mushroom growth without requiring farmers to take photographs using their mobile phones. These images are used as inputs for the trained YOLOv8 model.

2. Artificial Intelligence Layer

The AI layer uses YOLOv8, a deep learning-based object detection model, to identify four oyster mushroom growth-stage classes early growth, pinning, mature, and ready to harvest. The model analyzes visual characteristics in the captured images and generates detection results.

The trained model is evaluated using appropriate object-detection metrics. The reported mAP50 value is used to describe detection performance, while harvest-time estimation requires separate evaluation using suitable reference data.

3. Backend and Data Management Layer

Fast API provides the backend interface for connecting the AI model with the rest of the application. It receives image-processing requests, executes model inference, and returns the resulting information.

MongoDB manages relevant monitoring records, including rack identification, detection results, timestamps, and harvest-time estimates where available. This information supports the organization and retrieval of monitoring data.

Node-RED and MQTT can support communication and workflow integration between connected components. Their specific responsibilities depend on the devices and communication processes implemented in the final system.

4. Application Layer

React Native is used to develop the Android application through which farmers can access monitoring information. The application is intended to display rack-wise results and the estimated time remaining until harvest in a simple and understandable format.

The user interface is designed to emphasize harvesting information rather than requiring farmers to interpret technical AI outputs. This approach aims to make the system more accessible to small and medium-sized oyster mushroom farmers.

5.1.3 Technology Selection Rationale

The technologies were selected according to their suitability for image-based monitoring, system integration, and mobile application development.

YOLOv8 was selected because it supports object detection and localization in images. FastAPI provides a lightweight Python-based framework for exposing model inference through API endpoints. MongoDB supports flexible storage of monitoring records, while React Native allows the development of an Android application using a shared JavaScript based codebase.

Node RED and MQTT can support integration between connected devices and application services when these functions are required. Visual Studio Code provides a common development environment for managing the different software components.

Together, these technologies provide a foundation for integrating oyster mushroom image analysis, growth-stage detection, data management, and farmer-oriented monitoring into a single system.

5.2 Component implementation

The Biological Stage Detection and Harvest Prediction component was developed to automate the monitoring of oyster mushroom growth using artificial intelligence and computer vision. The main objective of this component is to identify mushroom growth stages from images and support the estimation of the remaining time until harvest. The implementation combines a trained YOLOv8 model with a backend service, database, and Android application to support practical mushroom monitoring.

5.2.1 Dataset Preparation and Preprocessing

The implementation began with preparing an image dataset for training the deep learning model. The dataset consisted of oyster mushroom images representing different growth stages. The images were labeled according to four classes: early growth, pinning, mature, and ready to harvest. These labels enabled the model to learn visual differences between the growth stages.

The dataset was prepared for YOLOv8 training by organizing the images and their corresponding annotations in the required format. The annotated images were used to train the model to identify mushrooms and predict their growth-stage classes. Dataset preparation was an important step because the quality and consistency of the annotations could affect the model's detection performance.

5.2.2 YOLOv8 Model Training

YOLOv8 was selected as the primary deep learning model for growth-stage detection. The model was trained using the prepared and labeled oyster mushroom images. During training, the model learned visual patterns such as mushroom size, cap shape, color, and cluster structure to distinguish between the defined growth-stage classes.

The model was trained for 50 epochs. Its performance was evaluated using object-detection metrics, including mean Average Precision at an Intersection over Union threshold of 0.50 (mAP50). The reported mAP50 was approximately 92.2%, indicating the model's ability to detect the labeled classes under the evaluation conditions.

This result provides an initial indication of the model's performance. However, it does not establish the accuracy of harvest-time prediction or guarantee the same performance under all real-world cultivation conditions. Further validation using independent images and practical farm environments is necessary to assess generalization.

5.2.3 Growth-Stage Detection

After training, the model was configured to identify four oyster mushroom growth-stage classes,

1. Early Growth: Mushrooms are beginning to develop.
2. Pinning: Small mushroom pins are visible.
3. Mature: Mushrooms have developed larger caps and more recognizable structures.
4. Ready to Harvest: Mushrooms appear to have reached the defined harvesting stage.

When an image is processed, the model predicts the locations and classes of the detected objects. The detection results can then be used by the wider system to monitor mushroom development.

The four class structure provides a consistent method of categorizing observed growth. However, the classification depends on the quality of the input image and the visual differences between the stages. Clear labeling guidelines and suitable validation data are therefore important for reliable detection.

5.2.4 Backend Implementation Using FastAPI

Fast API is used as the backend framework for integrating the trained YOLOv8 model with the wider application. The backend is responsible for receiving image-processing requests, passing images to the model, and returning the resulting detection information.

The intended processing workflow begins when an image is captured by the fixed camera. The image is then sent to the backend for inference. The backend processes the image using the trained model and returns information such as the detected growth-stage class, confidence score, and relevant detection coordinates.

The backend also provides the integration point for communicating with the database and mobile application. This design separates AI processing from the user interface and allows the model to be maintained independently of the Android application.

5.2.5 Database Implementation Using MongoDB

MongoDB is used to manage monitoring information associated with the mushroom growth-detection process. Relevant records may include rack identification, image timestamps, detected growth stages, confidence scores, and harvest-time estimates.

Organizing the results by rack and timestamp can support comparisons between observations taken at different times. These records can also help maintain a history of mushroom growth and provide reference information for evaluating the system.

The database structure should ensure that each record is associated with the correct rack and observation time. If harvest-time prediction is implemented, its estimates should be stored separately from the detection results so that the performance of both functions can be evaluated independently.

5.2.6 Harvest-Time Estimation

Harvest-time estimation is an important function of the proposed component. Growth-stage detection identifies the current visual category of a mushroom, whereas harvest-time estimation attempts to determine how much time remains before harvesting.

The detection results can provide useful inputs for this prediction process. However, the current growth stage alone may not be sufficient to produce reliable time estimates

because growth rates can vary according to cultivation conditions and individual mushroom development.

A suitable prediction method should therefore be developed using relevant growth observations and, where available, historical records of actual harvesting times. The resulting estimates should be compared with observed harvesting times to measure prediction error.

The system is intended to present the estimated time remaining until harvest in a simple format that farmers can understand. The reliability of this function must be assessed separately from the YOLOv8 model's detection performance.

5.2.7 Integration with the Android Application

React Native is used to develop the Android application that provides access to the monitoring results. The application communicates with the backend to retrieve the information required for presenting growth-monitoring and harvesting estimates.

The intended interface focuses on useful information for farmers, including rack identification and the estimated time remaining until harvest. Rather than requiring farmers to interpret technical model outputs, the application aims to present the results in a clear and accessible manner.

The application is designed to support rack-wise monitoring, allowing farmers to identify which cultivation areas may require attention. The usefulness of the interface depends on the successful integration of image processing, backend communication, database storage, and result presentation.

5.2.8 Overall Component Workflow

The overall implementation follows the workflow below:

1. A fixed camera captures images of oyster mushrooms at the cultivation racks.
2. The captured images are sent to the backend for processing.
3. The trained YOLOv8 model analyzes the images and identifies the growth-stage classes.
4. The detection results are returned to the backend.
5. Relevant monitoring records are stored in MongoDB.

6. The harvest-time estimation method processes the appropriate information, where implemented.
7. The backend provides the results to the React Native Android application.
8. The application presents the relevant rack information and estimated time remaining until harvest.

This modular workflow separates image capture, detection, prediction, data storage, and user interaction. It also allows individual components to be tested before evaluating the complete system.

5.3 Integration with the other components

5.4 Verification and software testing

Verification and software testing are important activities in the development of the Biological Stage Detection and Harvest Prediction component. The purpose of testing is to determine whether the implemented system functions according to its requirements and whether the individual components operate correctly. Testing focuses on the YOLOv8 growth-stage detection model, image-processing workflow, backend services, database operations, and integration with the Android application.

5.4.1 Model Verification

The YOLOv8 model was trained to identify four oyster mushroom growth-stage classes: early growth, pinning, mature, and ready to harvest. Model verification focuses on assessing whether the trained model can identify these classes from mushroom images.

The model was trained for 50 epochs, and its reported mean Average Precision at an Intersection over Union threshold of 0.50 (mAP50) was approximately 92.2%. This result provides an indication of the model's object-detection performance on the evaluation dataset.

Model verification should include testing with images that were not used during training. Precision, recall, and mAP50 can be used to evaluate detection performance. A confusion matrix may also help identify growth stages that are frequently misclassified. These evaluations are important because visually similar growth stages may create challenges for the model.

The reported mAP50 result should not be interpreted as the accuracy of harvest-time prediction. Harvest-time estimation requires separate testing using predicted harvesting times and actual harvesting records.

5.4.2 Functional Testing

Functional testing checks whether the system's features work according to their intended requirements. The main functions considered for this component include image processing, growth-stage detection, result storage, backend communication, and displaying monitoring information through the Android application.

The following table presents the main functional test cases.

Test ID	Test Case	Expected Result
FT01	Submit a valid oyster mushroom image for processing.	The system processes the image and returns the available detection results.
FT02	Submit an image containing mushrooms in the early growth stage.	The model attempts to identify the early growth class.
FT03	Submit an image containing mushrooms in the pinning stage.	The model attempts to identify the pinning class.
FT04	Submit an image containing mature mushrooms.	The model attempts to identify the mature class.
FT05	Submit an image containing mushrooms ready to harvest.	The model attempts to identify the ready-to-harvest class.
FT06	Submit an invalid or unsupported image file.	The system handles the error without crashing.
FT07	Store a detection result in MongoDB.	The relevant record is saved successfully when the database is available.

FT08	Retrieve monitoring records for a particular rack.	The system returns the records associated with the specified rack.
FT09	Request detection results through the backend API.	The API returns the appropriate response or an informative error.
FT10	View monitoring information through the Android application.	The application displays the available information correctly.
FT11	Request an estimated time remaining until harvest.	The system returns an estimate if the prediction functionality has been implemented and the required data are available.

The expected results in this table define the intended testing procedure. Actual test outcomes should be recorded after each test is performed.

5.4.3 Backend API Testing

Backend API testing verifies communication between the trained model, FastAPI service, database, and mobile application. The tests should confirm that valid requests are processed correctly and that the backend returns the expected response structure.

Testing should cover valid image submissions, missing input files, unsupported file formats, malformed requests, and situations where the database or model is unavailable. Appropriate error messages should be returned when a request cannot be processed.

The API should also be checked to ensure that detection results contain the required information, such as predicted class labels, confidence scores, and relevant detection coordinates. If rack identification and harvest-time estimates are supported, the response should include the appropriate information.

5.4.4 Database Testing

Database testing verifies that monitoring records are stored and retrieved correctly in MongoDB. Each record should contain the required information, such as rack identification, observation timestamp, detection results, and harvest-time estimates where available.

Testing should confirm that new records can be created, stored records can be retrieved, and observations from different racks remain correctly associated with their respective identifiers. The system should also handle database connection failures without terminating unexpectedly.

These tests help ensure that the monitoring history remains organized and that the Android application can retrieve the appropriate records.

5.4.5 Integration Testing

Integration testing examines whether the individual software components communicate correctly when combined. For this research component, integration testing focuses on the connection between the YOLOv8 model, FastAPI backend, MongoDB database, and React Native Android application.

The testing process should verify that an image can be submitted, processed by the model, and returned as a detection result. It should also verify that the result can be stored in the database and retrieved by the application.

If the fixed camera, micro-climate monitoring component, Node-RED, or MQTT are connected to the system, their communication and data-exchange functions should also be tested. Rack identifiers and timestamps should remain consistent when image-based observations are associated with environmental records.

5.4.6 Performance Testing

Performance testing evaluates whether the system can process images and return results within an acceptable time. Since the component is intended for practical mushroom farm monitoring, processing speed and system responsiveness are important considerations.

The planned performance tests include measuring image-processing time, API response time, and the time required to retrieve monitoring records. The previously defined image-processing target is less than three seconds. This target should be measured under clearly documented hardware, image-size, and network conditions.

The model's detection performance should also be evaluated on a separate test dataset. The reported mAP50 of approximately 92.2% provides an initial model evaluation

result, but further testing is necessary to determine how well the model performs on new images and under real cultivation conditions.

5.4.7 Harvest-Time Prediction Testing

Harvest-time prediction must be evaluated independently from growth-stage detection. The purpose of this testing is to determine whether the system can estimate the remaining time until harvesting with an acceptable level of error.

If the prediction method has been implemented, its estimates should be compared with the actual time remaining until harvest recorded during cultivation. Suitable evaluation measures may include Mean Absolute Error (MAE), which measures the average absolute difference between predicted and actual values.

Testing should include mushrooms observed at different growth stages and, where possible, across different cultivation racks and growing conditions. The results can help identify circumstances in which predictions are less reliable.

If harvest-time prediction has not yet been implemented or validated, it should be reported as a planned function rather than a verified capability.

5.4.8 User Interface Testing

User interface testing checks whether the Android application displays monitoring information correctly and allows users to access the available functions. Testing should examine the presentation of rack identifiers, detection results, and estimated harvesting times where supported.

The application should display appropriate messages when data are unavailable or the backend cannot be reached. The interface should also remain understandable for farmers who may not have technical knowledge of artificial intelligence.

User feedback can provide additional evidence about the clarity and usefulness of the application. If user testing is conducted with farmers, the number of participants, testing procedure, feedback, and findings should be documented accurately.

5.5 Deployment and operation

The deployment and operation of the Biological Stage Detection and Harvest Prediction component focus on making the developed system available for monitoring oyster mushroom growth. The component combines a fixed-camera image acquisition setup, a trained YOLOv8 model, a FastAPI backend, a MongoDB database, and a React Native Android application. These elements are intended to work together to identify mushroom growth stages and provide information to support harvesting decisions.

5.5.1 Deployment Configuration

The system is designed around a modular architecture in which image acquisition, AI processing, data storage, and user interaction operate as separate but connected components. The fixed camera is positioned near the oyster mushroom cultivation racks to capture images of mushroom development. The captured images are sent to the backend for processing by the trained YOLOv8 model.

The FastAPI backend manages image-processing requests and returns detection results. MongoDB stores relevant monitoring records, including rack identifiers, timestamps, detected growth stages, and harvest-time estimates where available. The React Native Android application provides the interface through which farmers can access the monitoring information.

The backend and database may be hosted on a suitable computer or server, depending on the deployment configuration. The selected environment should provide sufficient processing capacity, reliable network connectivity, and secure access to the required services.

5.5.2 Model Deployment

The trained YOLOv8 model is prepared for inference by making the trained model weights available to the backend service. When a valid image is received, the backend loads or accesses the model, processes the image, and returns the detected growth-stage classes and associated confidence scores.

The model was trained for 50 epochs and achieved a reported mAP50 of approximately 92.2%. This result provides an initial indication of detection performance on the

evaluation dataset. Before deployment in an actual mushroom farm, the model should also be tested using images captured under practical cultivation conditions.

The deployed model should use the same class definitions and compatible preprocessing settings as those used during training. Maintaining the correct model version and configuration helps ensure consistency between development and deployment.

5.5.3 System Operation

The normal operation of the component follows a sequence of image capture, processing, storage, and result presentation.

First, the fixed camera captures images of oyster mushrooms growing on the cultivation racks. Each image should be associated with the correct rack identifier and observation timestamp. The image is then submitted to the backend, where the trained YOLOv8 model identifies the relevant growth-stage classes.

The backend returns the detection results and stores the required monitoring information in MongoDB. Where the harvest-time estimation function has been implemented, the system also generates or retrieves an estimate of the remaining time until harvest. The Android application then displays the available information to the farmer.

The intended user interface emphasizes rack-wise monitoring and estimated harvesting time rather than requiring farmers to interpret technical model outputs. This workflow is designed to reduce the effort involved in repeatedly checking mushroom growth manually.

5.5.4 Monitoring and Maintenance

Regular monitoring and maintenance are necessary to maintain the reliability of the system. The backend service, database connection, camera operation, and image-processing workflow should be checked periodically to identify failures or interruptions.

The main maintenance activities include,

1. Checking that the fixed camera captures clear and usable images.
2. Verifying that the backend can receive and process image requests.

3. Monitoring the availability of the database and stored records.
4. Checking that detection results are associated with the correct racks and timestamps.
5. Reviewing model performance when new cultivation images become available.
6. Updating software components when necessary and retesting the system after changes.
7. Monitoring storage usage and removing or archiving data according to an appropriate retention policy.

If the model performs poorly on new images, additional labeled data may be required for retraining and evaluation. Any updated model should be tested before replacing the deployed version.

5.5.5 Data Backup and Recovery

Monitoring records are valuable because they can support the review of mushroom development over time. A suitable backup process should therefore be established for the MongoDB database and other important system data.

Database backups should be stored in a secure location separate from the primary database. The backup schedule should reflect how frequently new records are created and how much data could reasonably be lost following a failure.

Recovery procedures should be tested to confirm that backed-up records can be restored. Important files, including the trained model weights, model configuration, source code, and relevant deployment settings, should also be maintained in a controlled location.

Access to monitoring records and system services should be restricted to authorized users. These practices help protect the system against accidental data loss and unauthorized access.

5.5.6 Operational Limitations

The operational performance of the component may be affected by image quality, lighting conditions, camera placement, network availability, and variations in

mushroom appearance. Blurred images, overlapping mushroom clusters, or changes in the cultivation environment may reduce detection reliability.

The harvest-time estimation function may also be affected by differences in mushroom growth rates and cultivation conditions. Consequently, estimated harvesting times should be treated as decision-support information rather than guaranteed harvesting dates.

If the system depends on a remote backend, network interruptions may temporarily prevent image processing or result retrieval. Appropriate error handling, retry mechanisms, and clear status messages can help users understand when the system is unavailable.

6 RESULTS AND VALIDATION

6.1 Research results

The main research outcome of this component was the development of a deep learning-based approach for detecting the biological growth stages of oyster mushrooms from captured images. The research focused on four growth stages: early growth, pinning, mature, and ready to harvest.

A YOLOv8 object detection model was trained using labelled oyster mushroom images. The dataset was prepared by identifying and annotating mushroom images according to their relevant growth stages. The trained model was then evaluated to determine its ability to recognize the different stages.

The experimental results showed that the developed YOLOv8 model achieved approximately 92.2% mAP@50. This result indicates that the model was able to detect the target mushroom growth stages with a high level of detection performance under the tested conditions.

The results demonstrate that computer vision and deep learning can be used to support automated monitoring of oyster mushroom development. Instead of depending completely on manual visual observation, the proposed approach can analyze captured images and provide information about the current growth condition.

The research also considered the practical use of the detected growth information for harvest prediction. The detected biological condition can be used as an input for estimating the remaining time until harvesting. Therefore, the research does not focus only on recognizing mushroom stages but also on converting the recognition result into information that is more useful to farmers.

6.2 Product and technical results

The developed component consists of an image-based detection and prediction workflow. A fixed camera positioned at the cultivation rack captures mushroom images. These images are processed by the system and provided to the trained YOLOv8 model for biological stage detection.

The technical implementation includes a React Native Android application, FastAPI backend, MongoDB database, and the trained YOLOv8 model. The system architecture allows captured image information to be transferred to the backend, processed by the trained model, and returned as useful growth and harvest-related information.

The YOLOv8 model was trained for the four defined biological growth stages. During model evaluation, the model achieved approximately 92.2% mAP@50, which indicates that the trained model performed well in detecting the target classes.

The component also supports rack-level monitoring. Information can be associated with the relevant rack, allowing mushroom growth information to be organized according to the cultivation location.

An important product level result is the way the information is presented to the farmer. Instead of requiring the farmer to understand technical biological stage names, the system is designed to provide practical information such as the estimated time remaining until harvest or an indication that the mushrooms are ready for harvesting.

The developed approach therefore provides three main technical functions:

1. Capturing mushroom images using a fixed camera.

2. Detecting the biological growth condition using the YOLOv8 model.

3. Providing harvest-related information based on the detected growth condition.

These results demonstrate that the developed component can act as an intelligent image-analysis module within a larger smart mushroom farming system.

6.3 User or stakeholder validation

User validation was considered to determine whether the proposed component could provide useful information for its intended users. The main target users of the system are beginner and small to medium scale oyster mushroom farmers.

The validation focused on whether users could understand the information provided by the system and whether automated growth monitoring could support their normal farming activities. Particular attention was given to the harvest prediction information

because farmers are more interested in knowing when mushrooms are likely to be ready for harvesting rather than simply knowing the technical biological stage.

The proposed system presents the result in a simple and understandable form. For example, instead of displaying only a technical result such as a biological stage classification, the system can provide information such as Estimated time until harvest 2–3 days or Ready for harvest.

This approach reduces the need for users to interpret technical model outputs. It also makes the system more suitable for beginner farmers who may have limited experience in identifying mushroom growth stages.

The validation process also considers the practical usefulness of automated monitoring. A fixed-camera approach can reduce the need for farmers to manually capture images using their mobile phones every time they want to check mushroom growth.

The user validation therefore supports the suitability of the component as a farmer support tool. However, further validation with a larger number of farmers and across different cultivation environments would be useful to confirm the system's effectiveness under real-world conditions.

6.4 Integration results

The Biological Stage Detection & Harvest Prediction component was designed to operate as one part of the overall Rack Level Micro Climate & Growth Optimizer for Oyster Mushroom Farms.

The trained YOLOv8 model is connected to the backend system through the image-processing workflow. The FastAPI backend provides the communication layer between the application and the machine learning model. The detected information can then be stored and managed using MongoDB.

The component is also designed to work with rack-level information. This allows images and prediction results to be associated with specific mushroom cultivation racks. Therefore, the system can maintain a more organized record of mushroom growth instead of treating all mushrooms in the farm as a single group.

The integration also provides a foundation for combining biological growth information with other parts of the overall smart farming system. For example, growth information can potentially be considered together with environmental information such as temperature, humidity, CO_2, and other microclimate measurements.

The integration results demonstrate that the developed component is not an isolated machine learning model. It can operate as a software component within the wider smart agriculture system and communicate with other system components through the backend architecture.

6.5 Summary of findings

The results of this research demonstrate the feasibility of using YOLOv8-based computer vision for oyster mushroom biological stage detection and harvest prediction.

The trained model achieved approximately 92.2% mAP@50, showing strong detection performance for the defined mushroom growth stages. The results indicate that deep learning can provide an effective alternative to fully manual mushroom growth monitoring.

The developed component also provides practical harvest-related information by converting the detected growth condition into an estimated time until harvest or a ready for harvest indication. This makes the output more understandable and useful for farmers.

The system architecture successfully connects the machine learning model with the application and backend components using React Native, FastAPI, and MongoDB. The rack-level approach further supports organized monitoring of mushroom growth.

Overall, the findings indicate that the proposed Biological Stage Detection & Harvest Prediction component has the potential to support oyster mushroom farmers by providing automated, consistent, and understandable information about mushroom development and harvesting readiness. Nevertheless, additional real-world testing with larger datasets, more farms, different environmental conditions, and a larger group of farmers would strengthen the validation of the proposed system.

7 DISCUSSION

7.1 Interpretation of findings

The findings of this research show that the proposed Biological Stage Detection and Harvest Prediction component can support oyster mushroom farmers in monitoring mushroom growth more easily and accurately. The YOLOv8 model achieved approximately 92.2% mAP50, which indicates that the model was able to identify the different growth conditions of oyster mushrooms with a high level of accuracy. This result shows that image-based monitoring can be a useful method for reducing the need for continuous manual observation by farmers.

The results also indicate that the system can use visible mushroom characteristics such as size, shape, colour, and cap development to estimate the remaining time until harvest. This is important because farmers mainly need to know when the mushrooms are likely to be ready for harvesting, rather than only knowing the biological growth stage. Therefore, the proposed system provides more practical information for daily farm management.

The findings further suggest that using a fixed camera at rack level can provide more consistent monitoring compared with depending on farmers to manually take and upload images using their mobile phones. Rack-wise monitoring can also help identify differences in mushroom growth between different racks or growing areas. Overall, the findings support the idea that computer vision and machine learning can be applied to improve oyster mushroom growth monitoring and assist farmers in making better harvesting decisions.

7.2 Comparison with literature baselines and existing solutions

The proposed Biological Stage Detection and Harvest Prediction component can be compared with existing research and solutions that use computer vision, machine learning, and image processing for mushroom monitoring. Many existing approaches focus mainly on mushroom classification, disease detection, or identifying growth stages from images. These approaches demonstrate that visual characteristics such as mushroom size, shape, colour, and cap development can be used to monitor mushroom

growth. However, they do not always provide a direct prediction of the remaining time until harvest, which is the main practical focus of the proposed component.

Compared with traditional manual monitoring, the proposed system provides a more automated approach. In manual monitoring, farmers need to regularly observe each rack and decide whether mushrooms are ready for harvesting based on their experience. This can be time-consuming and may produce different decisions between farmers. The proposed system uses a fixed rack-level camera and YOLOv8 model to analyse mushroom images consistently. Therefore, it can reduce the dependence on continuous manual observation and provide more objective monitoring results.

Compared with existing image-based mushroom solutions, the proposed component also focuses on rack-level monitoring and harvest-time estimation. Instead of only displaying the detected biological stage, the system is designed to provide the farmer with useful information such as the estimated time remaining until the mushrooms are ready to harvest. This makes the output more suitable for practical farm decision-making.

The trained YOLOv8 model achieved approximately 92.2% mAP50, showing that the selected deep-learning approach can effectively detect the relevant mushroom growth characteristics in the collected dataset. This result provides evidence that the proposed approach is suitable for the intended oyster mushroom farming environment. However, direct numerical comparison with other studies should be made carefully because different studies may use different mushroom species, datasets, image conditions, model architectures, and evaluation metrics.

Overall, the comparison indicates that the proposed component extends the idea of existing computer-vision-based mushroom monitoring by combining automated image analysis, rack-level monitoring, and harvest prediction. This combination is particularly useful for beginner and small-to-medium oyster mushroom farmers who need simple and practical information to support harvesting decisions.

7.3 Research contribution

This research contributes to the development of a computer-vision-based biological growth monitoring and harvest prediction system for oyster mushroom farming. The main contribution is the application of a YOLOv8-based deep learning model to

identify oyster mushroom growth characteristics from images captured at rack level. The trained model achieved approximately 92.2% mAP50, demonstrating its ability to detect mushroom growth conditions with good accuracy.

Another important contribution is the focus on harvest-time prediction rather than only biological stage classification. The system is designed to convert the image analysis results into practical information for farmers, particularly the estimated time remaining until the mushrooms are ready to harvest. This can help farmers plan harvesting activities more effectively.

The research also contributes a rack-level automated monitoring approach using a fixed camera. This reduces the need for farmers to manually capture images using their mobile phones and allows growth information to be collected more consistently. The rack-wise approach can also support monitoring of differences in mushroom growth across different growing areas.

From a technical perspective, the research demonstrates how YOLOv8, React Native, FastAPI, MongoDB, Node-RED, and MQTT can be combined to develop an integrated monitoring solution. This provides a foundation for connecting image-based growth analysis with the wider micro-climate monitoring system.

Overall, the research contributes both technical and practical value by demonstrating that computer vision and machine learning can be used to support oyster mushroom farmers with automated growth monitoring and more informed harvesting decisions.

7.4 Product contribution

The main product contribution of this research is the development of a Biological Stage Detection and Harvest Prediction component for the proposed oyster mushroom farming system. The component provides an automated method for monitoring mushroom growth using images captured by a fixed camera positioned at the rack level. This reduces the need for farmers to manually check each mushroom rack continuously.

The product uses a YOLOv8 based computer vision model to analyse mushroom images based on visual characteristics such as size, shape, colour, and cap development. The analysed information is used to provide a practical estimated time until harvest. Instead of requiring farmers to understand technical biological growth

stages, the system presents information in a simple and useful form that can directly support harvesting decisions.

Another product contribution is the integration of the component with the wider smart farming system. The component uses FastAPI for backend processing, MongoDB for storing relevant records, and React Native for the mobile application interface. Node-RED and MQTT can support communication between the monitoring devices and the software system. This allows the component to operate as part of a larger rack-level monitoring solution.

The product is particularly designed for beginner and small-to-medium oyster mushroom farmers. By providing automated monitoring and harvest predictions, the system can help reduce manual monitoring effort, improve awareness of mushroom growth, and support better planning of harvesting activities. Therefore, the product contribution is not only the development of a machine-learning model but also the creation of a practical farmer-oriented solution that converts image analysis into useful harvesting information.

7.5 Limitations and threats to validity

Although the proposed Biological Stage Detection and Harvest Prediction component produced promising results, several limitations and threats to validity should be considered. First, the performance of the YOLOv8 model depends on the quality and diversity of the training dataset. If the dataset does not contain enough variations in mushroom size, colour, lighting conditions, growing environments, and growth patterns, the model may not perform equally well in all real-world farming conditions.

Another limitation is that the system is mainly designed for oyster mushrooms. Therefore, the results may not be directly applicable to other mushroom species because different species can have different growth characteristics and appearances. Similarly, the model may perform differently when used in farms with environmental conditions that are different from those represented in the training and testing data.

The fixed camera setup can also create limitations. Camera position, distance, image quality, lighting, and obstruction by other mushrooms may affect the quality of captured images. Poor lighting or partially hidden mushrooms can make accurate

detection more difficult. Therefore, maintaining a suitable camera position and image capturing environment is important for reliable operation.

The harvest prediction is also subject to natural biological variation. Mushroom growth can be affected by factors such as temperature, humidity, airflow, substrate condition, and other environmental conditions. Therefore, an image-based prediction may not always provide an exact harvest date. The prediction should be considered an estimated decision-support result rather than a guaranteed harvesting time.

There are also limitations related to the evaluation process. The achieved approximately 92.2% mAP50 result represents the performance of the model under the tested dataset and conditions. It does not necessarily guarantee the same performance in every real-world mushroom farm. Further testing with larger and more diverse datasets and different farming environments would be required to establish stronger generalisability.

Overall, these limitations do not invalidate the findings, but they indicate areas where further research and testing are required. Future improvements should focus on larger and more diverse datasets, different lighting and camera conditions, longer-term farm testing, and integration of additional environmental information to improve the reliability of harvest prediction.

7.6 Implications

The findings of this research have several important implications for oyster mushroom farming, smart agriculture, and future research. The proposed Biological Stage Detection and Harvest Prediction component demonstrates that computer vision and machine learning can be used to provide useful information for monitoring mushroom growth and planning harvesting activities.

For farmers, the system can reduce the need for continuous manual observation of mushroom racks. By providing an estimated time until harvest, farmers can better plan harvesting activities, labour requirements, and other farm operations. This can be particularly useful for beginner and small-to-medium-scale farmers who may not have extensive experience in identifying the appropriate harvesting time.

For farm management, automated rack-level monitoring can provide more consistent information about mushroom growth. Monitoring individual racks can help farmers identify differences in growth and pay attention to racks that may require additional observation. This can support better use of time and resources and improve the overall efficiency of farm operations.

From a smart agriculture perspective, the research demonstrates the potential of integrating artificial intelligence and computer vision into agricultural production. The use of a fixed camera, YOLOv8, FastAPI, MongoDB, React Native, Node-RED, and MQTT provides a foundation for developing more connected and automated mushroom farming systems.

The research also has implications for future research and development. The current system can be further improved by using larger and more diverse datasets, incorporating environmental data such as temperature and humidity, and testing the system across different mushroom farms. Combining visual growth information with micro-climate information could potentially produce more reliable harvest predictions.

Overall, the research indicates that AI-based monitoring can move oyster mushroom farming from mainly manual observation toward data-supported decision making. However, the system should be considered a decision-support tool for farmers, rather than a complete replacement for human judgement and practical farming experience.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This research focused on the development of Biological Stage Detection & Harvest Prediction as part of the overall Rack-Level Micro-Climate & Growth Optimizer for Oyster Mushroom Farms. The main purpose of the component was to develop an automated approach for identifying oyster mushroom biological growth stages from images and providing useful information about harvesting readiness.

The proposed solution uses computer vision and deep learning, particularly a YOLOv8-based object detection model, to identify four biological growth stages: early growth, pinning, mature, and ready to harvest. The system uses images captured from a fixed camera positioned at the mushroom cultivation rack. This approach reduces the need for continuous manual observation by farmers.

The developed model achieved approximately 92.2% mAP@50 during evaluation, demonstrating that the proposed approach can effectively detect the defined mushroom growth stages under the tested conditions. The detected growth information can then be used to provide a practical harvest-related output, such as the estimated time remaining until harvesting or an indication that the mushrooms are ready to harvest.

The component was also designed to integrate with the wider software system using React Native, FastAPI, and MongoDB. Therefore, the research demonstrates the feasibility of combining deep learning based image analysis with a practical agricultural monitoring application.

Overall, the research shows that an automated image-based approach can support oyster mushroom farmers by providing consistent and understandable information about mushroom development and harvesting readiness. Although further real-world testing is required, the results provide a strong foundation for the future development of intelligent mushroom growth monitoring systems.

8.2 Achievement of objectives

The objectives defined for the Biological Stage Detection & Harvest Prediction component were addressed through the research and development activities.

1. To collect and prepare suitable oyster mushroom images representing different biological growth stages.

This objective was addressed by preparing and organizing oyster mushroom image data representing the selected growth stages. The images were labelled according to their relevant biological stages and prepared for model training and evaluation.

2. To develop and train a YOLOv8-based deep learning model for detecting oyster mushroom growth stages.

This objective was successfully achieved. A YOLOv8 object detection model was trained to identify the defined oyster mushroom growth stages.

3. To evaluate the performance of the trained model.

This objective was achieved through model evaluation. The trained model achieved approximately 92.2% mAP@50, demonstrating strong detection performance under the tested conditions.

4. To integrate the trained model with a backend system for processing mushroom images.

This objective was addressed by integrating the machine learning model with the backend architecture using FastAPI.. The backend provides the communication between the application and the image-processing/model functionality.

5. To develop rack-level mushroom growth monitoring.

This objective was addressed by designing the system around fixed-camera image capture at the cultivation rack. This allows growth information to be associated with the relevant rack.

6. To estimate the remaining time until the mushrooms are ready for harvesting.

This objective was addressed by using the detected growth condition as a basis for providing harvest-related information. The system is designed to provide an estimated time until harvest or a ready-for-harvest indication.

7. To provide simple and understandable information to farmers.

This objective was considered in the design of the user-facing output. Instead of requiring farmers to understand technical biological stage classifications, the system

focuses on practical information such as estimated harvest time and harvesting readiness.

8. To evaluate the usefulness of the component for the target users.

This objective was addressed through user-oriented validation focusing on the understandability and practical usefulness of the system outputs for beginner and small-to-medium-scale oyster mushroom farmers.

Therefore, the major objectives of the research component were successfully addressed through the development, evaluation, and integration of the proposed system.

8.3 Individual research and product contributions

The main individual contribution of this research is the development of the Biological Stage Detection & Harvest Prediction component for oyster mushroom farming.

From a research perspective, the contribution is the investigation of how deep learning-based computer vision can be applied to identify biological growth stages of oyster mushrooms. The research also considers how stage detection can be connected to practical harvest prediction rather than treating stage classification as the final output.

The development and evaluation of the YOLOv8 model provide evidence that object detection techniques can be applied to this agricultural problem. The achieved approximately 92.2% mAP@50 demonstrates the potential of the proposed approach.

From a technical perspective, the individual contribution includes the preparation of image data, image annotation, YOLOv8 model training, model evaluation, image-processing workflow, and integration of the trained model with the application backend.

From a product perspective, the component provides a practical farmer oriented solution for monitoring mushroom growth. The fixed camera approach supports rack level monitoring, while the harvest-related output converts technical model predictions into information that can be more easily understood by farmers.

The component also contributes to the overall project by providing biological growth information that can potentially be combined with environmental and microclimate information from other project components. This creates an opportunity for more comprehensive decision support for oyster mushroom farming.

8.4 Future directions

Although the developed component achieved promising results, several improvements can be considered for future development.

1. Larger and More Diverse Dataset

Future work should use a larger dataset containing images from different farms, mushroom varieties, lighting conditions, camera positions, and cultivation environments. A more diverse dataset could improve the generalization ability of the model.

2. Improved Harvest Prediction

The current approach can be further developed into a more accurate time-to-harvest prediction model. Future research could combine biological stage information with environmental variables such as temperature, humidity, CO_2 concentration, airflow, and substrate conditions.

3. Continuous Growth Monitoring

Instead of analyzing individual images independently, future versions could analyze images captured at regular time intervals. This would allow the system to track the growth pattern of mushrooms over time and improve harvest prediction.

4. Improved Camera and Edge Deployment

The system could be extended to operate directly on an edge device located near the cultivation rack. This could reduce communication delays and allow image processing even when internet connectivity is limited.

5. Real-Time Notifications

Future versions could provide notifications when mushrooms are approaching the harvesting stage. For example, farmers could receive an alert when the estimated harvest period is reached.

6. Multi-Rack and Multi-Bag Monitoring

The component could be expanded to identify individual racks and cultivation bags more accurately. This would allow farmers to monitor the growth status of multiple racks and bags separately.

7. Integration with Microclimate Prediction

A major future direction is the integration of biological growth information with the project's microclimate monitoring component. Combining mushroom growth predictions with temperature, humidity, CO_2, airflow, and moisture data could allow the system to identify relationships between environmental conditions and mushroom development.

8. Further User Validation

Future studies should involve a larger number of oyster mushroom farmers and conduct longer-term testing in real farming environments. This would provide stronger evidence about the usability, reliability, and practical benefits of the system.

9. Model Improvement

Different YOLO versions, computer vision techniques, and deep learning architectures could be compared in future research. Model optimization could also be performed to improve detection accuracy while reducing processing time and computational requirements.

In conclusion, the developed Biological Stage Detection & Harvest Prediction component provides a strong foundation for intelligent oyster mushroom growth monitoring. Future improvements can transform the current prototype into a more robust, real-time, and widely deployable agricultural decision support solution.

REFERENCES

[1] L. Guo, W. E. Zhang, W. Chen, N. Yang, Q. Nguyen, and T. D. Vo, "Oyster mushroom growth stage identification: An exploration of computer vision technologies," in *Advances in Artificial Intelligence*, Lecture Notes in Computer Science, vol. 14471, pp. 67–78, 2023, doi: 10.1007/978-981-99-8388-9_6.

[2] "Mushroom growth stage detection and maturity prediction using lightweight vision and logistic models for smart cultivation," *Smart Agricultural Technology*, vol. 13, Art. no. 101798, Mar. 2026, doi: 10.1016/j.atech.2026.101798.

[3] C. Charisis, S. Nuwayhid, and D. Argyropoulos, "A novel Mask R-CNN-based tracking pipeline for oyster mushroom cluster growth monitoring in time-lapse image datasets," *Computers and Electronics in Agriculture*, vol. 237, Art. no. 110590, Oct. 2025, doi: 10.1016/j.compag.2025.110590.

[4] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: Ultralytics YOLOv8 Documentation.

[5] Ultralytics, "YOLOv8 models," *Ultralytics Documentation*. [Online]. Available: YOLOv8 model documentation.

[6] FastAPI, "Tutorial – User Guide," *FastAPI Documentation*. [Online]. Available: FastAPI Documentation.

[7] MongoDB, "MongoDB Documentation." [Online]. Available: MongoDB Documentation.

[8] OASIS, "MQTT Version 5.0," *OASIS Standard*, Mar. 2019. [Online]. Available: MQTT Version 5.0 Specification.

[9] Meta Open Source, "React Native Documentation." [Online]. Available: React Native Documentation.

[10] Kaggle, "MushR Raw Image Dataset." [Online]. Available: Kaggle.

GLOSSARY

- Biological Growth Stage – A specific phase in the development of an oyster mushroom, from early growth to maturity and readiness for harvesting.
- Computer Vision – A technology that enables computers to analyse and understand information from images or videos.
- Deep Learning – A type of machine learning that uses multiple layers of artificial neural networks to learn patterns from data.
- Dataset – A collection of images or other information used to train, test, and evaluate a machine learning model.
- Harvest Prediction – The process of estimating how much time remains before oyster mushrooms are ready to be harvested.
- Image Annotation – The process of labelling images to identify and describe the objects or growth stages shown in them.
- Image Processing – The use of computer techniques to analyse and extract useful information from digital images.
- Machine Learning Model – A computer system trained using data to identify patterns and make predictions.
- Mean Average Precision (mAP) – An evaluation measure used to assess the performance of an object detection model by considering its precision across different classes.
- Oyster Mushroom – An edible mushroom belonging to the genus Pleurotus, commonly cultivated for food and commercial purposes.
- Object Detection – A computer vision task that identifies objects in an image and determines their locations.
- Rack-Level Monitoring – The process of monitoring mushroom growth and related information separately for each cultivation rack.
- Real-Time Monitoring – The process of observing and collecting information about a system as events occur.
- Training Data – The data used to teach a machine learning model to recognise patterns and make predictions.
- YOLOv8 – A version of the You Only Look Once object detection model family used to identify and locate objects in images.

APPENDICES

[Guidance - delete before submission: Include supplementary evidence such as instruments, ethics documents, detailed results, additional test cases, selected technical specifications, and AI-use disclosure. Do not use appendices to avoid explaining essential material in the main thesis.]

Appendix A Research instruments or extended results

[Guidance - delete before submission: Replace the title and content as appropriate, and cite the appendix from the thesis.]

Appendix B Academic integrity and AI use disclosure

[Guidance - delete before submission: List the tools used, purpose, representative prompts or workflows where required, verification performed, and sections or artefacts affected, in accordance with module policy.]